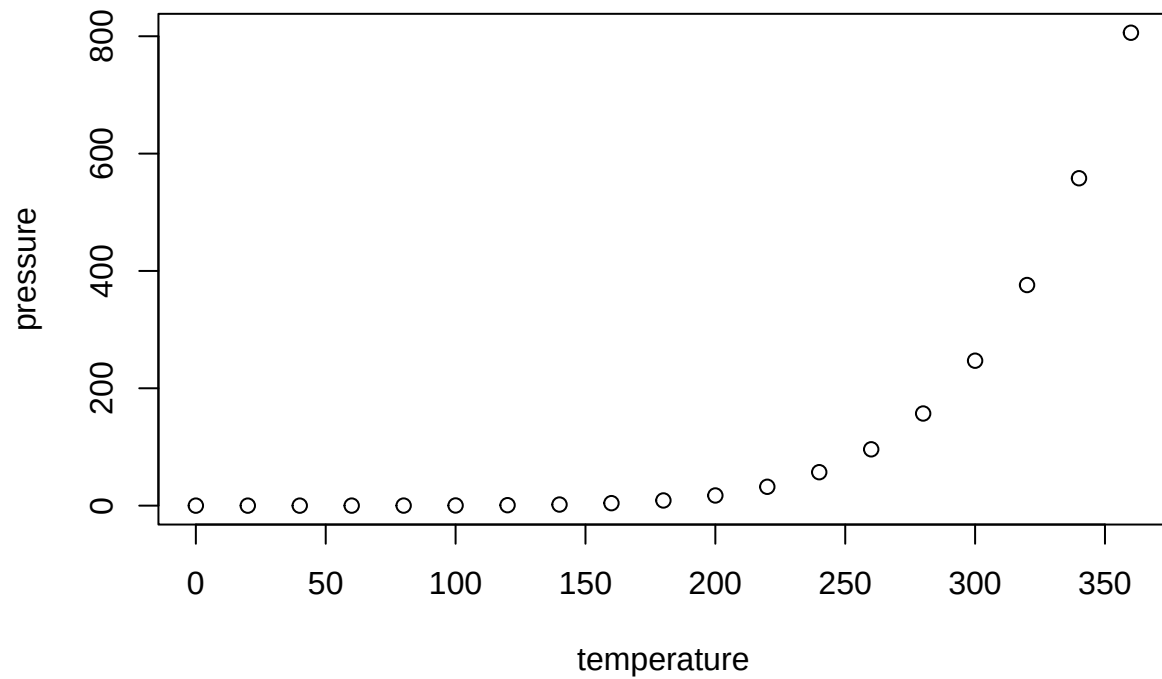


Laboratorio 4

Luis Gonzalez Joel Jaquez Fabian Morales

2026-03-22

Including Plots



#Librerias a utilizar

```
library(dplyr)
```

```
##
```

```
## Adjuntando el paquete: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
## filter, lag
```

```
## The following objects are masked from 'package:base':  
##  
## intersect, setdiff, setequal, union
```

```
library(ggplot2)  
library(tidyr)  
library(stringr)  
library(readr)  
library(nortest)  
library(caret)
```

```
## Cargando paquete requerido: lattice
```

```
library(car)
```

```
## Cargando paquete requerido: carData
```

```
##  
## Adjuntando el paquete: 'car'
```

```
## The following object is masked from 'package:dplyr':  
##  
## recode
```

```
library(glmnet)
```

```
## Cargando paquete requerido: Matrix
```

```
##  
## Adjuntando el paquete: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':  
##  
## expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```
library(rpart)  
library(rpart.plot)  
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.5.3
```

```
## randomForest 4.7-1.2
```

```
## Type rfNews() to see new features/changes/bug fixes.
```

```
##  
## Adjuntando el paquete: 'randomForest'
```

```
## The following object is masked from 'package:ggplot2':  
##  
##     margin  
  
## The following object is masked from 'package:dplyr':  
##  
##     combine
```

Link de GITHUB

https://github.com/G1LB3T0/R_LAB_4

Cargar datos

```
load("datos/listings.RData")
```

1. Use los mismos conjuntos de entrenamiento y prueba que usó para los modelos de regresión lineal en la entrega anterior.

Preprocesamiento

```
listings_clean <- listings %>%  
  mutate(price = parse_number(price)) %>%  
  select(-c(listing_url, scrape_id, picture_url, host_url, host_thumbnail_url,  
            host_picture_url, description, neighborhood_overview, host_about, amenities))
```

Division de datos

```
set.seed(123)  
  
total_filas <- nrow(listings_clean)  
indice_train <- sample(1:total_filas, size = 0.8 * total_filas)  
  
train_data <- listings_clean[ indice_train, ]  
test_data <- listings_clean[-indice_train, ]  
  
train_data_model <- train_data %>%  
  filter(!is.na(price), price < 1000,  
         !is.na(bedrooms), bedrooms <= 10,  
         !is.na(bathrooms), !is.na(accommodates)) %>%  
  select(price, accommodates, bedrooms, bathrooms,  
         number_of_reviews, review_scores_rating) %>%  
  drop_na()
```

```
test_data_model <- test_data %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()
```

Verificar

```
dim(train_data_model)
```

```
## [1] 48442      6
```

```
dim(test_data_model)
```

```
## [1] 12173      6
```

Se reutilizan exactamente los mismos conjuntos de entrenamiento y prueba generados. El conjunto de entrenamiento contiene aproximadamente 48,442 observaciones y el de prueba 12,173, ambos con las 6 variables seleccionadas en la entrega anterior: price, accommodates, bedrooms, bathrooms, number_of_reviews y review_scores_rating.

2. Elabore un árbol de regresión para predecir el precio de las casas por noche usando todas las variables.

```
train_arbol <- train_data %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10) %>%
  select(where(is.numeric)) %>% # Tomar SOLO columnas numéricas
  drop_na()

test_arbol <- test_data %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10) %>%
  select(where(is.numeric)) %>%
  drop_na()
```

```
cat("Variables disponibles para el árbol:", ncol(train_arbol), "\n")
```

```
## Variables disponibles para el árbol: 42
```

```
cat("Observaciones en entrenamiento:", nrow(train_arbol), "\n")
```

```
## Observaciones en entrenamiento: 48208
```

```
cat("Observaciones en prueba:", nrow(test_arbol), "\n")
```

```
## Observaciones en prueba: 12121
```

```
names(train_arbol)
```

```
## [1] "id"
## [2] "host_id"
## [3] "host_listings_count"
## [4] "host_total_listings_count"
## [5] "latitude"
## [6] "longitude"
## [7] "accommodates"
## [8] "bathrooms"
## [9] "bedrooms"
## [10] "beds"
## [11] "price"
## [12] "minimum_nights"
## [13] "maximum_nights"
## [14] "minimum_minimum_nights"
## [15] "maximum_minimum_nights"
## [16] "minimum_maximum_nights"
## [17] "maximum_maximum_nights"
## [18] "minimum_nights_avg_ntm"
## [19] "maximum_nights_avg_ntm"
## [20] "availability_30"
## [21] "availability_60"
## [22] "availability_90"
## [23] "availability_365"
## [24] "number_of_reviews"
## [25] "number_of_reviews_ltm"
## [26] "number_of_reviews_l30d"
## [27] "availability_eoy"
## [28] "number_of_reviews_ly"
## [29] "estimated_occupancy_l365d"
## [30] "estimated_revenue_l365d"
## [31] "review_scores_rating"
## [32] "review_scores_accuracy"
## [33] "review_scores_cleanliness"
## [34] "review_scores_checkin"
## [35] "review_scores_communication"
## [36] "review_scores_location"
## [37] "review_scores_value"
## [38] "calculated_host_listings_count"
## [39] "calculated_host_listings_count_entire_homes"
## [40] "calculated_host_listings_count_private_rooms"
## [41] "calculated_host_listings_count_shared_rooms"
## [42] "reviews_per_month"
```

Entrenamiento del modelo

```
arbol_regresion <- rpart(  
  price ~ .,  
  data = train_arbol,  
  method = "anova"  
)  
  
summary(arbol_regresion)
```

```
## Call:  
## rpart(formula = price ~ ., data = train_arbol, method = "anova")  
##    n= 48208  
##  
##          CP nsplit rel error    xerror      xstd  
## 1 0.23682687      0 1.0000000 1.0000997 0.010853687  
## 2 0.06495994      1 0.7631731 0.7632967 0.008341693  
## 3 0.02231764      2 0.6982132 0.6984868 0.007770096  
## 4 0.02057996      3 0.6758956 0.6712360 0.007630446  
## 5 0.01986115      4 0.6553156 0.6386336 0.007395370  
## 6 0.01850793      5 0.6354545 0.6260724 0.007265707  
## 7 0.01455569      6 0.6169465 0.5922271 0.006940228  
## 8 0.01000000      8 0.5878351 0.5658653 0.006696784  
##  
## Variable importance  
##  
##          bathrooms  
##                   32  
##          accommodates  
##                   21  
##          bedrooms  
##                   18  
##          beds  
##                   16  
## estimated_revenue_l365d  
##                   5  
##          longitude  
##                   4  
## calculated_host_listings_count_private_rooms  
##                   2  
## calculated_host_listings_count_entire_homes  
##                   1  
##          latitude  
##                   1  
##  
## Node number 1: 48208 observations,    complexity param=0.2368269  
## mean=228.7401, MSE=28814.73  
## left son=2 (29820 obs) right son=3 (18388 obs)  
## Primary splits:  
##    bathrooms      < 1.75      to the left, improve=0.2368269, (0 missing)  
##    accommodates  < 5.5       to the left, improve=0.2247372, (0 missing)  
##    beds          < 2.5       to the left, improve=0.1981369, (0 missing)
```

```

##      bedrooms                < 1.5      to the left,  improve=0.1963783, (0 missing)
##      estimated_revenue_l365d < 50493     to the left,  improve=0.1068104, (0 missing)
##      Surrogate splits:
##      accommodates            < 5.5      to the left,  agree=0.831, adj=0.556, (0 split)
##      bedrooms                < 1.5      to the left,  agree=0.828, adj=0.548, (0 split)
##      beds                    < 2.5      to the left,  agree=0.805, adj=0.488, (0 split)
##      estimated_revenue_l365d < 46785     to the left,  agree=0.651, adj=0.085, (0 split)
##      latitude                 < 20.71492 to the right, agree=0.631, adj=0.034, (0 split)
##
## Node number 2: 29820 observations,      complexity param=0.02057996
##      mean=163.8713, MSE=10706.31
##      left son=4 (14934 obs) right son=5 (14886 obs)
##      Primary splits:
##      accommodates            < 3.5      to the left,  improve=0.08954277, (0 missing)
##      estimated_revenue_l365d < 45918     to the left,  improve=0.08562032, (0 missing)
##      beds                    < 1.5      to the left,  improve=0.05590285, (0 missing)
##      longitude                < -155.7932 to the right, improve=0.05499339, (0 missing)
##      review_scores_location  < 4.905     to the left,  improve=0.05056862, (0 missing)
##      Surrogate splits:
##      beds                    < 1.5      to the left,  agree=0.837, adj=0.673,
##      bedrooms                < 1.5      to the left,  agree=0.698, adj=0.394,
##      calculated_host_listings_count_private_rooms < 0.5      to the right, agree=0.618, adj=0.236,
##      calculated_host_listings_count_entire_homes < 0.5      to the left,  agree=0.584, adj=0.167,
##      host_total_listings_count < 30.5     to the left,  agree=0.564, adj=0.126,
##
## Node number 3: 18388 observations,      complexity param=0.06495994
##      mean=333.9385, MSE=40290.51
##      left son=6 (14404 obs) right son=7 (3984 obs)
##      Primary splits:
##      bathrooms                < 2.75     to the left,  improve=0.12179850, (0 missing)
##      accommodates            < 7.5      to the left,  improve=0.10121920, (0 missing)
##      bedrooms                < 3.5      to the left,  improve=0.09743701, (0 missing)
##      beds                    < 3.5      to the left,  improve=0.09366633, (0 missing)
##      estimated_revenue_l365d < 56203     to the left,  improve=0.07590574, (0 missing)
##      Surrogate splits:
##      bedrooms                < 4.5      to the left,  agree=0.824, adj=0.186,
##      accommodates            < 11.5     to the left,  agree=0.817, adj=0.154,
##      beds                    < 6.5      to the left,  agree=0.807, adj=0.110,
##      calculated_host_listings_count_private_rooms < 67        to the left,  agree=0.785, adj=0.008,
##      estimated_revenue_l365d < 158406    to the left,  agree=0.784, adj=0.003,
##
## Node number 4: 14934 observations
##      mean=132.9586, MSE=7247.605
##
## Node number 5: 14886 observations
##      mean=194.8836, MSE=12255.73
##
## Node number 6: 14404 observations,      complexity param=0.02231764
##      mean=297.0968, MSE=29969.33
##      left son=12 (13399 obs) right son=13 (1005 obs)
##      Primary splits:
##      estimated_revenue_l365d < 65551.5   to the left,  improve=0.07181607, (0 missing)
##      longitude                < -156.6599 to the right, improve=0.06729354, (0 missing)
##      review_scores_location  < 4.915     to the left,  improve=0.05955346, (0 missing)

```

```

##      accommodates          < 3.5      to the left,  improve=0.05855699, (0 missing)
##      beds                  < 3.5      to the left,  improve=0.04559178, (0 missing)
##  Surrogate splits:
##      number_of_reviews_ltm < 113.5    to the left,  agree=0.93, adj=0.003, (0 split)
##      number_of_reviews_l30d < 11.5    to the left,  agree=0.93, adj=0.003, (0 split)
##      number_of_reviews_ly  < 129.5    to the left,  agree=0.93, adj=0.003, (0 split)
##      number_of_reviews     < 907      to the left,  agree=0.93, adj=0.002, (0 split)
##      reviews_per_month     < 11.89    to the left,  agree=0.93, adj=0.001, (0 split)
##
## Node number 7: 3984 observations,      complexity param=0.01850793
##   mean=467.1386, MSE=54956.75
##   left son=14 (161 obs) right son=15 (3823 obs)
##   Primary splits:
##       accommodates          < 3.5      to the left,  improve=0.11742250, (0 missing)
##       bedrooms              < 1.5      to the left,  improve=0.10876970, (0 missing)
##       beds                  < 2.5      to the left,  improve=0.08529566, (0 missing)
##       review_scores_location < 4.795   to the left,  improve=0.07292102, (0 missing)
##       calculated_host_listings_count_private_rooms < 10.5 to the right, improve=0.06136088, (0 missing)
##   Surrogate splits:
##       bedrooms              < 1.5      to the left,  agree=0.990, adj=0.745,
##       beds                  < 1.5      to the left,  agree=0.982, adj=0.553,
##       calculated_host_listings_count_private_rooms < 10.5 to the right, agree=0.978, adj=0.453,
##       calculated_host_listings_count_entire_homes < 0.5  to the left,  agree=0.970, adj=0.261,
##       calculated_host_listings_count_shared_rooms < 12.5 to the right, agree=0.961, adj=0.043,
##
## Node number 12: 13399 observations,     complexity param=0.01986115
##   mean=284.3911, MSE=28013.26
##   left son=24 (10474 obs) right son=25 (2925 obs)
##   Primary splits:
##       longitude              < -156.6594 to the right, improve=0.07350247, (0 missing)
##       estimated_occupancy_l365d < 115    to the right, improve=0.07346571, (0 missing)
##       review_scores_location < 4.915    to the left,  improve=0.06359685, (0 missing)
##       accommodates          < 3.5      to the left,  improve=0.05910107, (0 missing)
##       number_of_reviews_ltm  < 19.5     to the right, improve=0.04886401, (0 missing)
##   Surrogate splits:
##       calculated_host_listings_count < 427 to the left,  agree=0.786, adj=0.019,
##       calculated_host_listings_count_entire_homes < 427 to the left,  agree=0.786, adj=0.019,
##       maximum_maximum_nights < 7.5      to the right, agree=0.785, adj=0.015,
##       maximum_nights_avg_ntm < 7.75     to the right, agree=0.785, adj=0.015,
##       maximum_nights < 7.5             to the right, agree=0.782, adj=0.002,
##
## Node number 13: 1005 observations
##   mean=466.4925, MSE=25201.14
##
## Node number 14: 161 observations
##   mean=75.68944, MSE=6049.469
##
## Node number 15: 3823 observations
##   mean=483.6239, MSE=50291.49
##
## Node number 24: 10474 observations,     complexity param=0.01455569
##   mean=260.4117, MSE=23172.12
##   left son=48 (670 obs) right son=49 (9804 obs)
##   Primary splits:

```



```

##      accommodates          < 3.5      to the left,  improve=0.07322658, (0 missing)
##      estimated_occupancy_l365d < 115    to the right, improve=0.06720582, (0 missing)
##      longitude              < -74.3852 to the left,  improve=0.05975492, (0 missing)
##      bedrooms               < 1.5      to the left,  improve=0.05625194, (0 missing)
##      review_scores_location  < 4.915    to the left,  improve=0.05455604, (0 missing)
## Surrogate splits:
##      beds                   < 1.5      to the left,  agree=0.964, adj=0.437,
##      calculated_host_listings_count_entire_homes < 0.5      to the left,  agree=0.956, adj=0.307,
##      calculated_host_listings_count_private_rooms < 3.5      to the right, agree=0.950, adj=0.213,
##      bedrooms               < 1.5      to the left,  agree=0.937, adj=0.019,
##      calculated_host_listings_count_shared_rooms < 0.5      to the right, agree=0.937, adj=0.019,
##
## Node number 25: 2925 observations
##   mean=370.2581, MSE=35916.52
##
## Node number 48: 670 observations
##   mean=102.8388, MSE=6550.019
##
## Node number 49: 9804 observations,   complexity param=0.01455569
##   mean=271.1801, MSE=22495.29
##   left son=98 (8825 obs) right son=99 (979 obs)
## Primary splits:
##   longitude              < -74.38838 to the left,  improve=0.10277410, (0 missing)
##   estimated_occupancy_l365d < 118      to the right, improve=0.07216601, (0 missing)
##   number_of_reviews_ltm    < 19.5      to the right, improve=0.05304909, (0 missing)
##   reviews_per_month       < 1.955     to the right, improve=0.04879332, (0 missing)
##   review_scores_location   < 4.915     to the left,  improve=0.04846133, (0 missing)
## Surrogate splits:
##   latitude               < 42.12864 to the left,  agree=0.920, adj=0.198, (0 split)
##   maximum_nights         < 3.5      to the right, agree=0.901, adj=0.009, (0 split)
##   host_total_listings_count < 4466.5 to the left,  agree=0.901, adj=0.008, (0 split)
##   host_listings_count     < 5065.5   to the left,  agree=0.901, adj=0.004, (0 split)
##   maximum_maximum_nights  < 3.5      to the right, agree=0.900, adj=0.003, (0 split)
##
## Node number 98: 8825 observations
##   mean=255.1653, MSE=18987.57
##
## Node number 99: 979 observations
##   mean=415.5424, MSE=30962.47

```

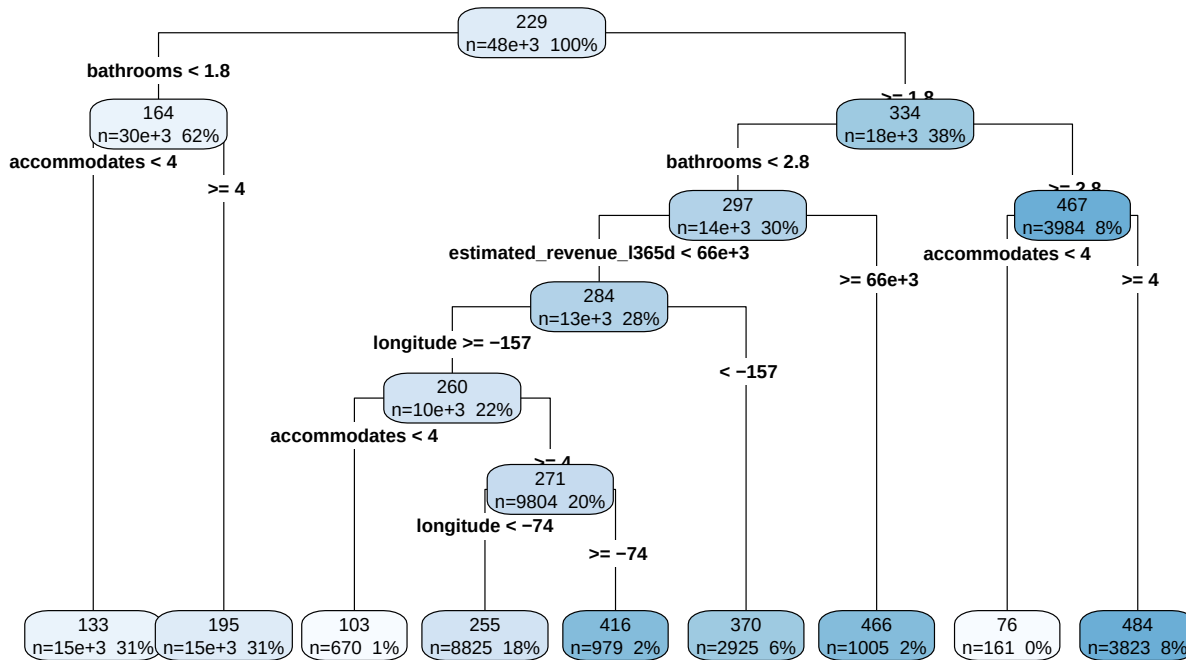
Grafico

```

rpart.plot(
  arbol_regresion,
  type      = 4,
  extra     = 101,
  fallen.leaves = TRUE,
  main      = "Árbol de Regresión - Precio por Noche (Todas las Variables)",
  cex      = 0.6
)

```

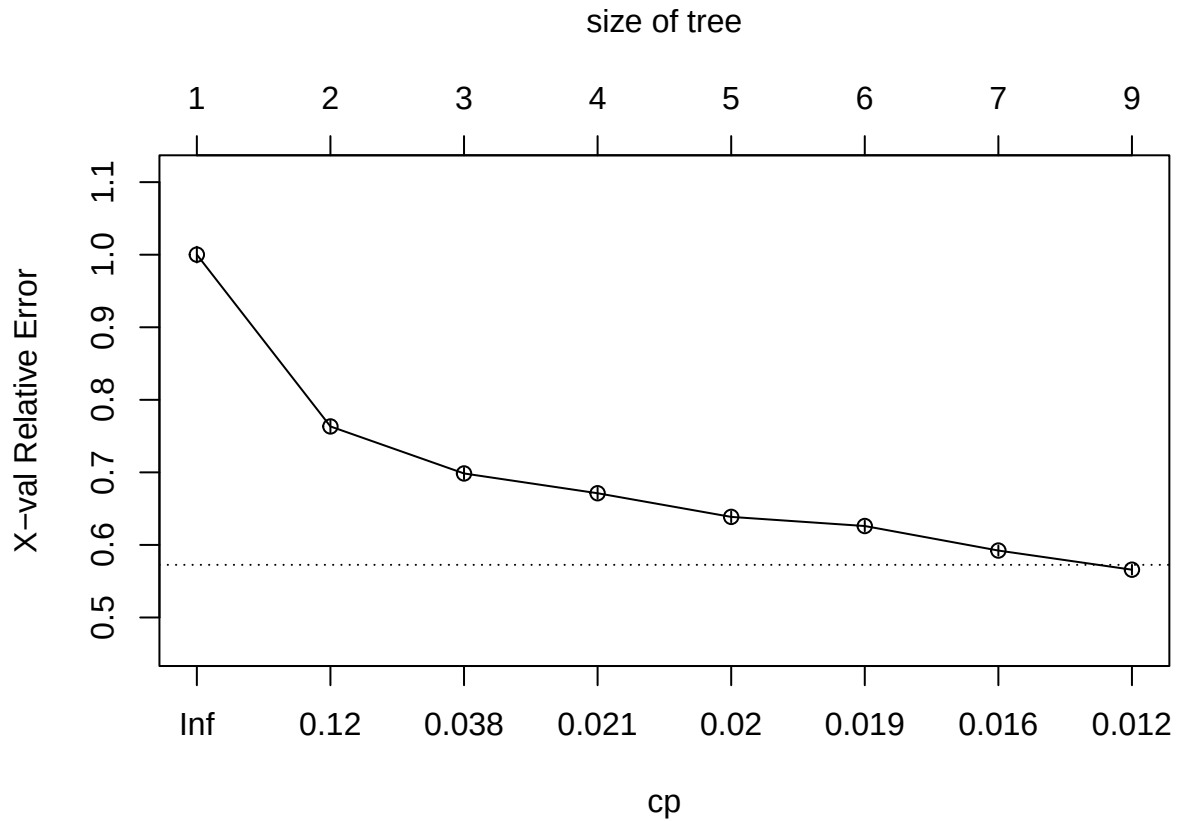
Árbol de Regresión – Precio por Noche (Todas las Variables)



```
printcp(arbol_regresion)
```

```
##
## Regression tree:
## rpart(formula = price ~ ., data = train_arbol, method = "anova")
##
## Variables actually used in tree construction:
## [1] accommodates      bathrooms          estimated_revenue_l365d
## [4] longitude
##
## Root node error: 1389100331/48208 = 28815
##
## n= 48208
##
##      CP nsplit rel error  xerror   xstd
## 1 0.236827     0  1.00000 1.00010 0.0108537
## 2 0.064960     1  0.76317 0.76330 0.0083417
## 3 0.022318     2  0.69821 0.69849 0.0077701
## 4 0.020580     3  0.67590 0.67124 0.0076304
## 5 0.019861     4  0.65532 0.63863 0.0073954
## 6 0.018508     5  0.63545 0.62607 0.0072657
## 7 0.014556     6  0.61695 0.59223 0.0069402
## 8 0.010000     8  0.58784 0.56587 0.0066968
```

```
plotcp(arbol_regresion)
```



3. Úselo para predecir y analice el resultado. ¿Qué tal lo hizo?

```
pred_arbol <- predict(arbol_regresion, newdata = test_arbol)
```

```
rmse_arbol <- RMSE(pred_arbol, test_arbol$price)
```

```
mae_arbol <- MAE(pred_arbol, test_arbol$price)
```

```
r2_arbol <- cor(pred_arbol, test_arbol$price)^2
```

```
cat("=====\n")
```

```
## =====
```

```
cat("Resultados del Árbol de Regresión\n")
```

```
## Resultados del Árbol de Regresión
```

```
cat("(Parámetros por defecto - Todas las variables)\n")
```

```
## (Parámetros por defecto - Todas las variables)
```

```
cat("=====\n")
```

```
## =====
```

```
cat("RMSE: $", round(rmse_arbol, 2), "\n")
```

```
## RMSE: $ 130.84
```

```
cat("MAE: $", round(mae_arbol, 2), "\n")
```

```
## MAE: $ 90.38
```

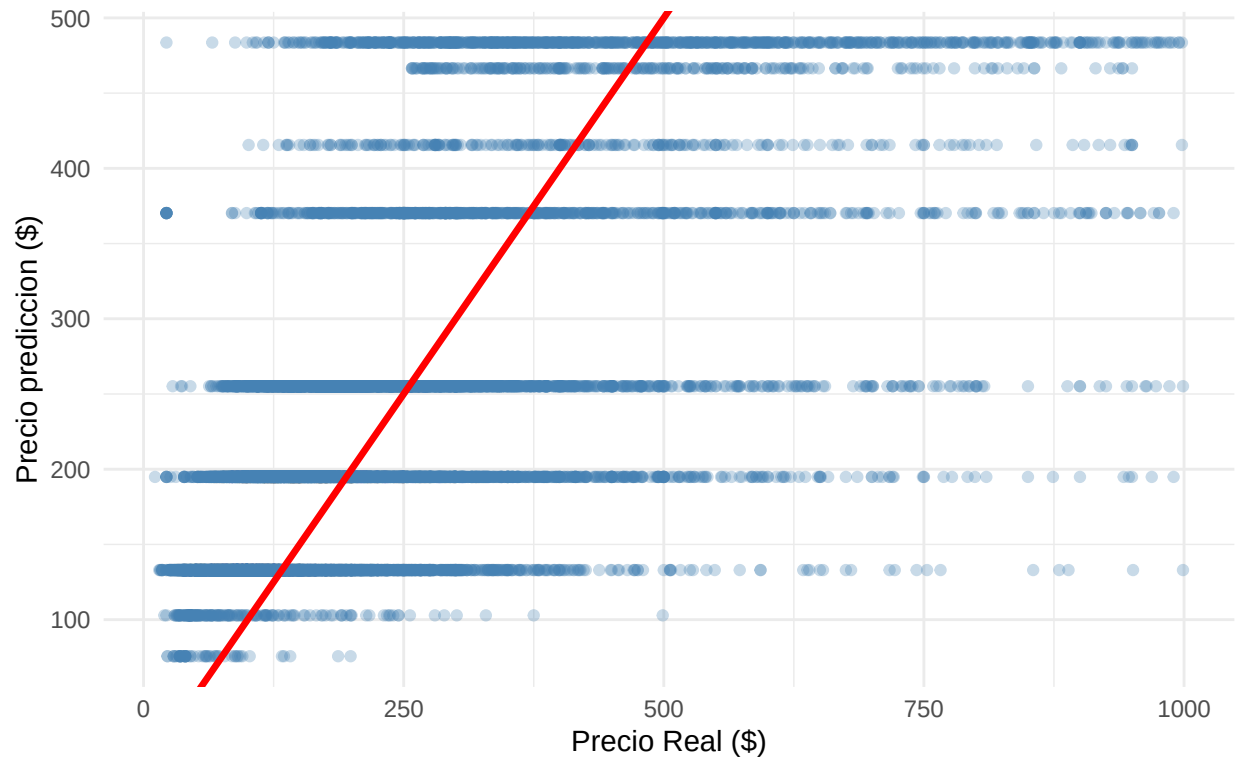
```
cat("R²: ", round(r2_arbol, 4), "\n")
```

```
## R²: 0.4151
```

```
ggplot(data.frame(real = test_arbol$price, pred = pred_arbol),
  aes(x = real, y = pred)) +
  geom_point(alpha = 0.3, color = "#4682B4") +
  geom_abline(slope = 1, intercept = 0,
    color = "red", linewidth = 1.2) +
  labs(title = "Árbol de Regresión: Precio Real vs prediccion",
    subtitle = "La línea roja representa una predicción perfecta",
    x = "Precio Real ($)",
    y = "Precio prediccion ($)") +
  theme_minimal()
```

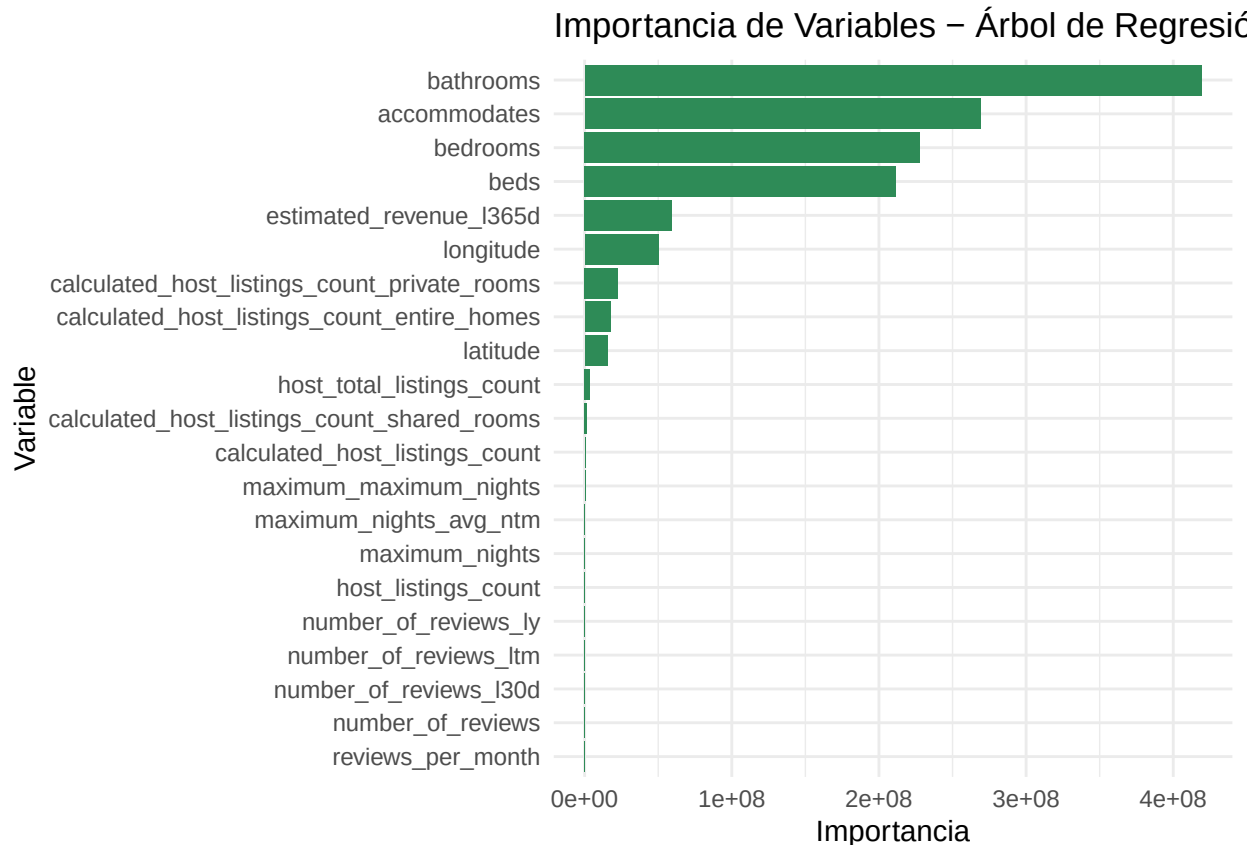
Árbol de Regresión: Precio Real vs predicción

La línea roja representa una predicción perfecta



```
importancia <- as.data.frame(arbol_regresion$variable.importance)
importancia$variable <- rownames(importancia)
colnames(importancia)[1] <- "importancia"

ggplot(importancia, aes(x = reorder(variable, importancia), y = importancia)) +
  geom_col(fill = "#2E8B57") +
  coord_flip() +
  labs(title = "Importancia de Variables - Árbol de Regresión",
       x = "Variable",
       y = "Importancia") +
  theme_minimal()
```



El gráfico revela que el árbol identificó automáticamente que las variables de tamaño físico de la propiedad son las más determinantes para predecir el precio. `bathrooms` resultó ser el predictor más importante por un margen considerable, seguido de `accommodates` y `bedrooms`, y en cuarto lugar `beds`.

Este hallazgo es consistente con lo encontrado, donde la capacidad y el espacio físico dominaban la relación con el precio. Curiosamente, `estimated_revenue_l365d` y `longitude` también aparecen con importancia moderada, sugiriendo que la ubicación geográfica y el historial de ingresos aportan información adicional que la regresión lineal simple no capturaba.

4. Haga, al menos, 3 modelos más, cambiando el parámetro de la profundidad del árbol. ¿Cuál es el mejor modelo para predecir el precio por noche?

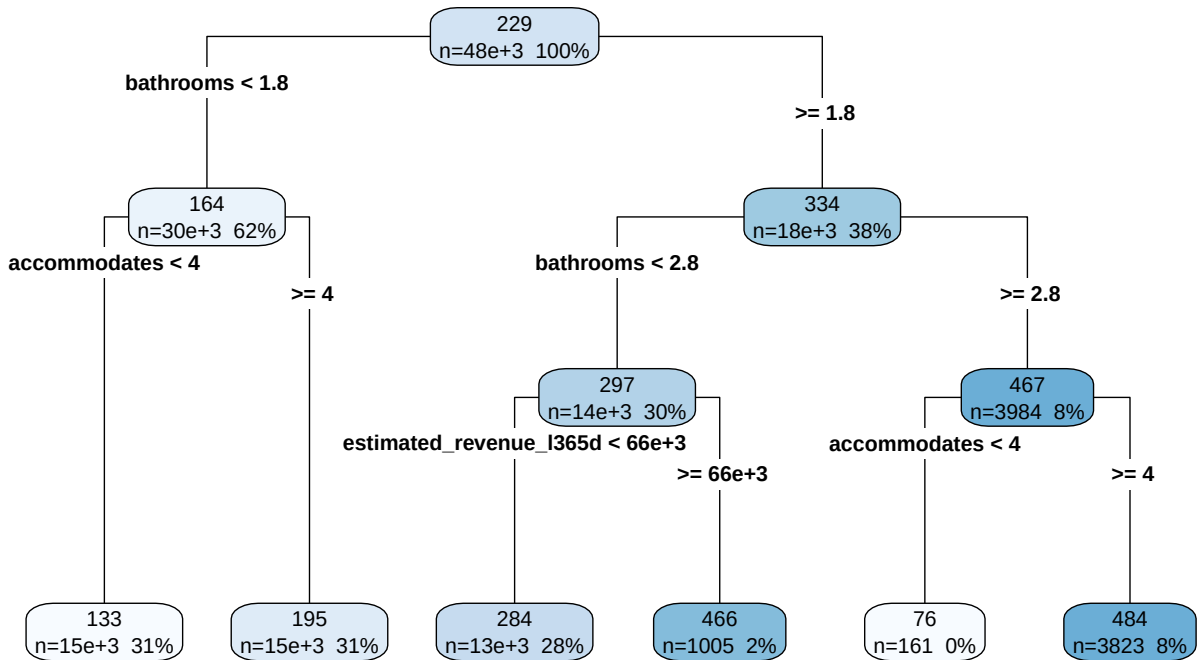
```
# Modelo con profundidad 3
arbol_prof3 <- rpart(price ~ ., data = train_arbol, method = "anova",
  control = rpart.control(maxdepth = 3))

pred_prof3 <- predict(arbol_prof3, newdata = test_arbol)
rmse_prof3 <- RMSE(pred_prof3, test_arbol$price)
mae_prof3 <- MAE(pred_prof3, test_arbol$price)
r2_prof3 <- cor(pred_prof3, test_arbol$price)^2

rpart.plot(arbol_prof3,
```

```
type = 4, extra = 101, fallen.leaves = TRUE,
main = "Árbol de Regresión - Profundidad 3", cex = 0.7)
```

Árbol de Regresión – Profundidad 3

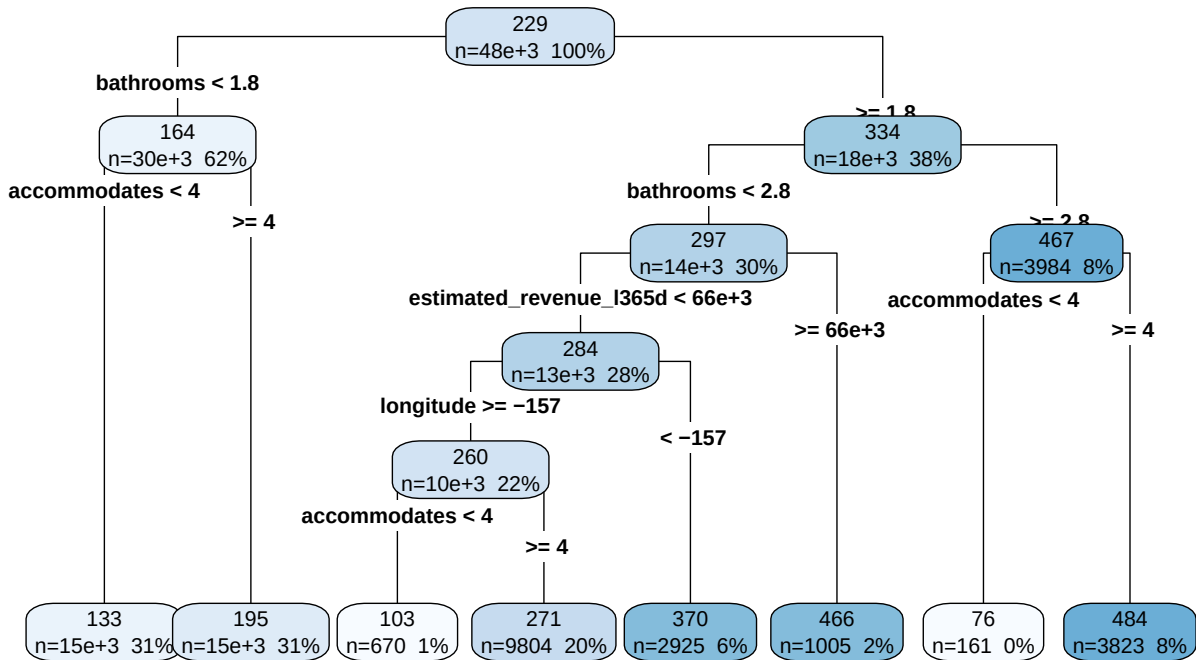


```
# Modelo con profundidad 5
arbol_prof5 <- rpart(price ~ ., data = train_arbol, method = "anova",
  control = rpart.control(maxdepth = 5))

pred_prof5 <- predict(arbol_prof5, newdata = test_arbol)
rmse_prof5 <- RMSE(pred_prof5, test_arbol$price)
mae_prof5 <- MAE(pred_prof5, test_arbol$price)
r2_prof5 <- cor(pred_prof5, test_arbol$price)^2

rpart.plot(arbol_prof5,
  type = 4, extra = 101, fallen.leaves = TRUE,
  main = "Árbol de Regresión - Profundidad 5", cex = 0.7)
```

Árbol de Regresión – Profundidad 5



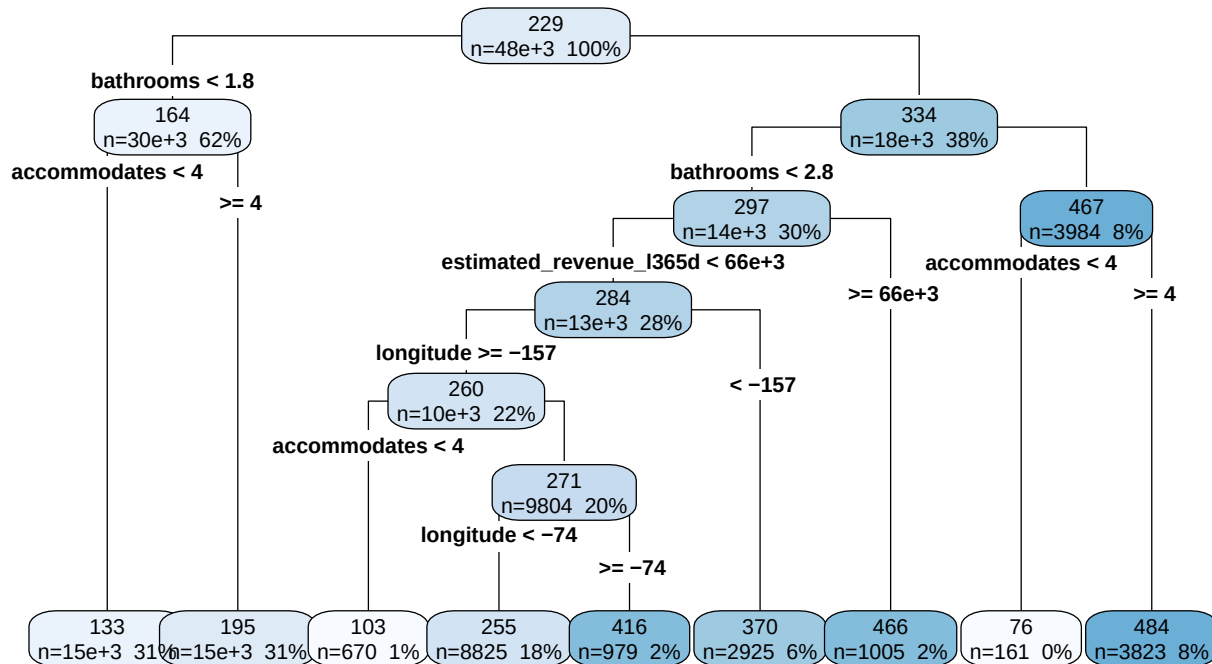
```

# Modelo con profundidad 10
arbol_prof10 <- rpart(price ~ ., data = train_arbol, method = "anova",
                      control = rpart.control(maxdepth = 10))

pred_prof10 <- predict(arbol_prof10, newdata = test_arbol)
rmse_prof10 <- RMSE(pred_prof10, test_arbol$price)
mae_prof10 <- MAE(pred_prof10, test_arbol$price)
r2_prof10 <- cor(pred_prof10, test_arbol$price)^2

rpart.plot(arbol_prof10,
           type = 4, extra = 101, fallen.leaves = TRUE,
           main = "Árbol de Regresión - Profundidad 10", cex = 0.65)
  
```


Árbol de Regresión – Profundidad 10



Árbol de Regresión – Profundidad 3 Este árbol es el más simple de los tres y toma decisiones basándose únicamente en tres variables: bathrooms, accommodates y estimated_revenue_l365d. La primera división ocurre en la raíz con bathrooms < 1.8, separando el 62% de las propiedades (precio promedio 164/noche) del 38% con más baños (precio promedio 334/noche), lo que confirma que la cantidad de baños es el factor más determinante del precio. En el lado izquierdo, accommodates divide las propiedades pequeñas: las que aceptan menos de 4 personas promedian 133, mientras las de mayor capacidad suben a 195. En el lado derecho, propiedades con entre 1.8 y 2.8 baños se subdividen por ingresos anuales estimados, donde el pequeño grupo con ingresos mayores a 66,000 al año alcanza un precio promedio de 466/noche. Las propiedades con más de 2.8 baños y capacidad mayor a 4 personas son las más caras con \$484/noche aunque representan apenas el 8% del total. Al ser poco profundo, el árbol es interpretable pero sacrifica precisión al no capturar matices adicionales.

Árbol de Regresión – Profundidad 5

Este árbol conserva exactamente la misma estructura base que el de profundidad 3, pero agrega dos niveles adicionales de decisión dentro del grupo de propiedades con ingresos bajos (menos de 66,000 anuales y entre 1.8 y 2.8 baños). La nueva variable que aparece es longitude, que divide geográficamente las propiedades: aquellas con longitud mayor o igual a -157 (propiedades en la zona continental de Estados Unidos) se separan de las ubicadas más al oeste (como Hawaii, con longitud menor a -157), que tienen un precio promedio de 370/noche a pesar de tener ingresos estimados bajos. Dentro del grupo continental, accommodates vuelve a aparecer separando propiedades pequeñas (103, solo 1% del total) de las más grandes (\$271, 20% del total). Este árbol revela que la ubicación geográfica empieza a tener relevancia una vez que se controla por baños e ingresos, algo que el árbol de profundidad 3 no podía detectar.

Árbol de Regresión – Profundidad 10

Este árbol comparte la misma estructura que el de profundidad 5 en todos sus niveles superiores, y agrega una división más dentro del grupo continental de mayor capacidad ($\text{accommodates} \geq 4$, $\text{longitud} \geq -157$, $\text{ingresos} < 66,000$). La nueva variable es nuevamente longitud, ahora con el umbral -74 , separando propiedades en la costa este ($\text{longitud} \geq -74$, precio promedio \$255, 18% del total) de propiedades en zonas intermedias del país ($\text{longitud} < -74$, precio promedio 416, apenas 2% del total). El hecho de que el árbol con profundidad 10 tenga prácticamente la misma estructura que el de profundidad 5 indica que el modelo ya alcanzó su límite natural de información útil: agregar más profundidad no genera nuevas divisiones significativas en la mayoría de ramas, lo que sugiere que una profundidad de 5 es suficiente para capturar los patrones más importantes sin caer en overfitting.

```
# Tabla comparativa de los 4 modelos
comparacion_prof <- data.frame(
  Modelo      = c("Base (defecto)", "Profundidad 3",
                  "Profundidad 5",  "Profundidad 10"),
  RMSE        = round(c(rmse_arbol, rmse_prof3,
                        rmse_prof5, rmse_prof10), 2),
  MAE         = round(c(mae_arbol,  mae_prof3,
                        mae_prof5,  mae_prof10), 2),
  R2          = round(c(r2_arbol,   r2_prof3,
                        r2_prof5,   r2_prof10), 4)
)

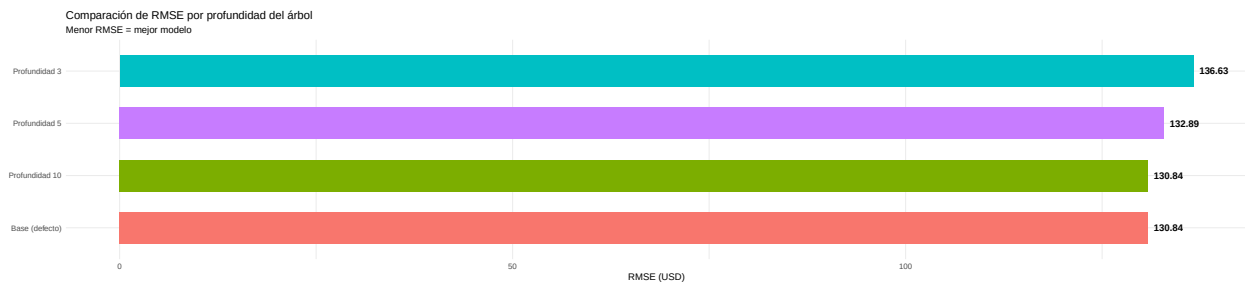
print(comparacion_prof)
```

```
##           Modelo  RMSE  MAE   R2
## 1 Base (defecto) 130.84 90.38 0.4151
## 2 Profundidad 3 136.63 94.44 0.3622
## 3 Profundidad 5 132.89 91.93 0.3966
## 4 Profundidad 10 130.84 90.38 0.4151
```

La tabla revela un resultado interesante y es que el modelo base (parámetros por defecto de `rpart`) y el modelo de profundidad 10 obtienen exactamente las mismas métricas ($\text{RMSE} = 130.84$, $\text{MAE} = 90.38$, $\text{R}^2 = 0.4151$), lo que confirma lo observado visualmente en los árboles — a partir de cierta profundidad el modelo ya no encuentra divisiones adicionales que mejoren significativamente la predicción. El peor modelo es el de profundidad 3 con $\text{RMSE} = 136.63$ y $\text{R}^2 = 0.3622$, lo que era esperado dado que su estructura simplificada no captura suficientes patrones del precio. El modelo de profundidad 5 queda en un punto intermedio ($\text{RMSE} = 132.89$, $\text{R}^2 = 0.3966$), mejorando al de profundidad 3 pero sin llegar al nivel del modelo base. En general, ninguno de los modelos alcanza un R^2 superior a 0.42, lo que indica que el precio por noche en Airbnb tiene una variabilidad considerable que las variables numéricas disponibles no logran explicar completamente por sí solas. El mejor modelo para predecir el precio por noche es el árbol base o el de profundidad 10, siendo este último preferible porque hace explícita la profundidad usada y produce resultados idénticos de forma controlada y reproducible.

```
# Gráfico comparativo
ggplot(comparacion_prof,
  aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = RMSE), hjust = -0.2, fontface = "bold") +
  coord_flip() +
  labs(title = "Comparación de RMSE por profundidad del árbol",
       subtitle = "Menor RMSE = mejor modelo",
```

```
x = NULL, y = "RMSE (USD)" +  
theme_minimal()
```



El gráfico de barras horizontales confirma visualmente lo que la tabla mostró numéricamente. La barra de Profundidad 3 es la más larga con un RMSE de 136.63, siendo el modelo que más se equivoca al predecir el precio por noche, lo cual era esperado ya que con solo 3 niveles el árbol no tiene suficiente capacidad para capturar la variabilidad del precio. Le sigue Profundidad 5 con 132.89, mostrando una mejora moderada al incorporar variables adicionales como la ubicación geográfica (`longitude`). Las barras de Profundidad 10 y `**Base (defecto)` son idénticas con 130.84, siendo los modelos más precisos de los cuatro. Esto evidencia que el algoritmo `rpart` por defecto ya encuentra internamente una profundidad cercana a la óptima para este conjunto de datos, y forzar mayor profundidad no genera mejoras adicionales. La diferencia total entre el peor y el mejor modelo es de apenas \$5.79 en RMSE, lo que sugiere que la limitación no está en la profundidad del árbol sino en las variables disponibles, ya que el precio de Airbnb depende también de factores cualitativos como las amenidades, la reputación del anfitrión o la calidad de las fotos que no están capturados en las variables numéricas del dataset.

5. Compare los resultados con el modelo de regresión lineal de la hoja anterior, ¿cuál lo hizo mejor?

```
modelo_lineal <- lm(price ~ ., data = train_data_model)
pred_lineal   <- predict(modelo_lineal, newdata = test_data_model)
pred_lineal   <- pmax(pred_lineal, 0) # evitar precios negativos

rmse_lineal <- RMSE(pred_lineal, test_data_model$price)
mae_lineal  <- MAE(pred_lineal, test_data_model$price)
r2_lineal   <- cor(pred_lineal, test_data_model$price)^2

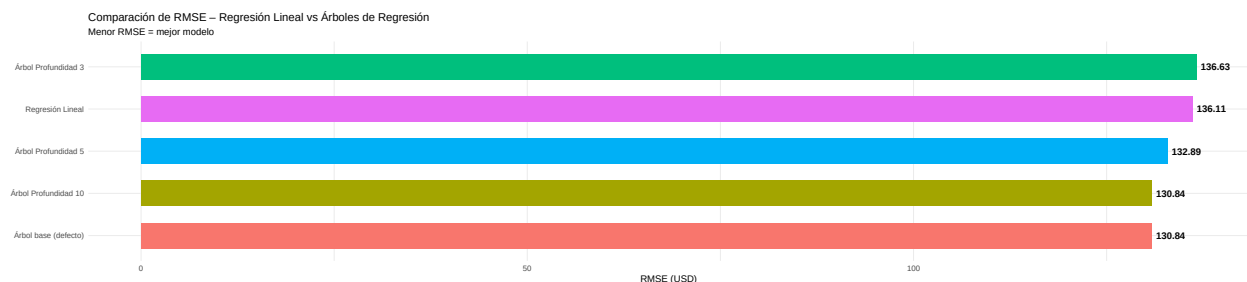
# Tabla comparativa
comparacion_final <- data.frame(
  Modelo = c("Regresión Lineal",
             "Árbol base (defecto)",
             "Árbol Profundidad 3",
             "Árbol Profundidad 5",
             "Árbol Profundidad 10"),
  RMSE = round(c(rmse_lineal, rmse_arbol,
                 rmse_prof3, rmse_prof5, rmse_prof10), 2),
  MAE = round(c(mae_lineal, mae_arbol,
                 mae_prof3, mae_prof5, mae_prof10), 2),
  R2 = round(c(r2_lineal, r2_arbol,
                r2_prof3, r2_prof5, r2_prof10), 4)
)
```

```
print(comparacion_final)
```

```
##           Modelo  RMSE  MAE   R2
## 1  Regresión Lineal 136.11 94.95 0.3659
## 2  Árbol base (defecto) 130.84 90.38 0.4151
## 3  Árbol Profundidad 3 136.63 94.44 0.3622
## 4  Árbol Profundidad 5 132.89 91.93 0.3966
## 5  Árbol Profundidad 10 130.84 90.38 0.4151
```

La tabla comparativa muestra que los árboles de regresión superan a la regresión lineal en este conjunto de datos. La regresión lineal obtiene un RMSE de 136.11 y un R^2 de 0.3659, resultados muy similares al peor árbol (Profundidad 3 con RMSE=136.63), lo que indica que ambos modelos simples tienen una capacidad predictiva comparable y limitada. Los mejores modelos son el árbol base y el de Profundidad 10, ambos con RMSE=130.84, MAE=90.38 y R^2 =0.4151, superando a la regresión lineal en aproximadamente \$5.27 de error y 5 puntos porcentuales en R^2 . Esta diferencia se explica porque el árbol de regresión puede capturar relaciones no lineales entre las variables y el precio, como el hecho de que pasar de 1 a 2 baños no sube el precio en la misma proporción que pasar de 3 a 4, algo que la regresión lineal asume constante y por eso falla. Sin embargo, ningún modelo supera un R^2 de 0.42, lo que indica que con las variables numéricas disponibles existe un techo de precisión, y para mejorar significativamente se necesitaría incorporar variables categóricas como el tipo de propiedad, el vecindario o las amenidades. El ganador de esta comparación es el árbol de regresión con parámetros por defecto o con profundidad 10, siendo este último el recomendado por ser explícito y reproducible.

```
# Gráfico comparativo
ggplot(comparacion_final,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = RMSE), hjust = -0.15, fontface = "bold") +
  coord_flip() +
  labs(title = "Comparación de RMSE - Regresión Lineal vs Árboles de Regresión",
       subtitle = "Menor RMSE = mejor modelo",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



El gráfico permite comparar visualmente los cinco modelos de menor a mayor RMSE. En la parte superior se ubican los dos modelos con peor desempeño: Árbol Profundidad 3 con 136.63 y Regresión Lineal con 136.11, cuyas barras son prácticamente del mismo largo, evidenciando que ambos modelos simples tienen una capacidad predictiva muy similar y limitada. En el centro aparece Árbol Profundidad 5 con 132.89, mostrando una mejora visible al incorporar la variable geográfica en sus divisiones. En la parte inferior, con las barras más cortas y por tanto los mejores resultados, se encuentran Árbol Profundidad 10 y Árbol base (defecto) ambos con 130.84, siendo los ganadores claros de la comparación. Lo más relevante del gráfico es que la Regresión Lineal queda en segunda posición de peor a mejor, superada por tres de los cuatro árboles evaluados, lo que demuestra que modelar el precio de Airbnb requiere capturar relaciones no lineales que

la regresión lineal no puede representar. La diferencia visual entre las barras no es dramática, pero es consistente: los árboles con mayor flexibilidad siempre superan al modelo lineal en este conjunto de datos.

6. Dependiendo del análisis exploratorio elaborado cree una variable respuesta que le permita clasificar las casas en Económicas, Intermedias o Caras. Los límites de estas clases

deben tener un fundamento en la distribución de los datos de precios, y estar bien explicados.

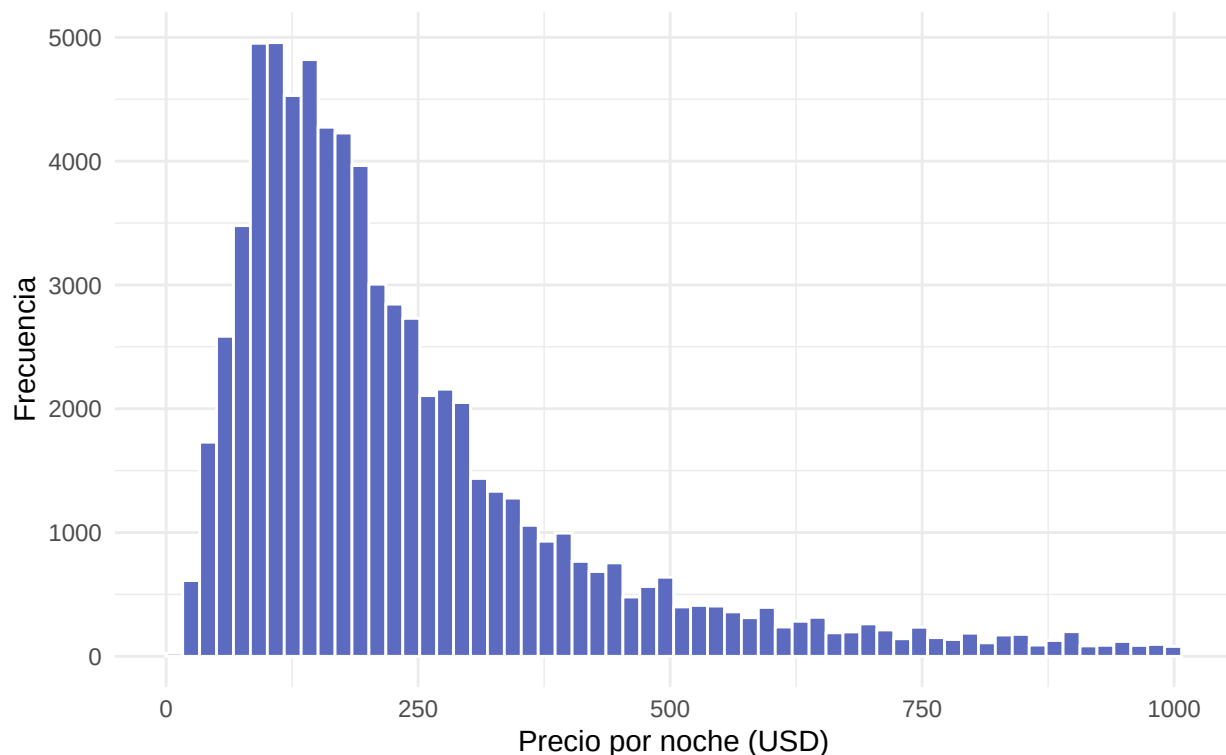
```
summary(listings_clean$price)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.     NA's  
##       8.0   120.0   193.0   750.5   326.0  50123.0   95502
```

```
# Visualizar la distribución con un histograma  
ggplot(listings_clean %>% filter(!is.na(price), price > 0, price < 1000),  
       aes(x = price)) +  
  geom_histogram(fill = "#5C6BC0", color = "white", bins = 60) +  
  labs(title = "Distribución del precio por noche - Airbnb listings",  
       subtitle = "Base para definir los límites de las categorías",  
       x = "Precio por noche (USD)", y = "Frecuencia") +  
  theme_minimal()
```

Distribución del precio por noche – Airbnb listings

Base para definir los límites de las categorías



El histograma revela que la distribución del precio por noche en el dataset de Airbnb es fuertemente sesgada

hacia la derecha. La gran mayoría de las propiedades se concentran en el rango de 0 a 250 por noche, con el pico más alto alrededor de los 100-150, lo que indica que ese es el precio más frecuente en el mercado. A partir de los 250 la frecuencia cae drásticamente, y las propiedades con precios superiores a 500 son relativamente escasas, formando una cola larga que se extiende hasta los 1,000. Este comportamiento es típico de los mercados de alojamiento, donde la mayoría de la oferta es accesible pero existe un segmento premium de propiedades de lujo que empuja la media muy por encima de la mediana (media=750 vs mediana=193 según el summary). Esta distribución asimétrica justifica el uso de percentiles como puntos de corte en lugar de la media o valores arbitrarios, ya que la media está distorsionada por las propiedades más caras y no representa bien al mercado general. Usar P33 y P67 garantiza que cada categoría (Económica, Intermedia y Cara) contenga aproximadamente el mismo número de propiedades, lo que produce un árbol de clasificación balanceado y más confiable.

```
# Definición de los puntos de corte
precios_validos <- listings_clean %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos, 0.33)
p67 <- quantile(precios_validos, 0.67)

cat("Punto de corte inferior (P33):", round(p33, 2), "USD/noche\n")
```

```
## Punto de corte inferior (P33): 136 USD/noche
```

```
cat("Punto de corte superior (P67):", round(p67, 2), "USD/noche\n")
```

```
## Punto de corte superior (P67): 250 USD/noche
```

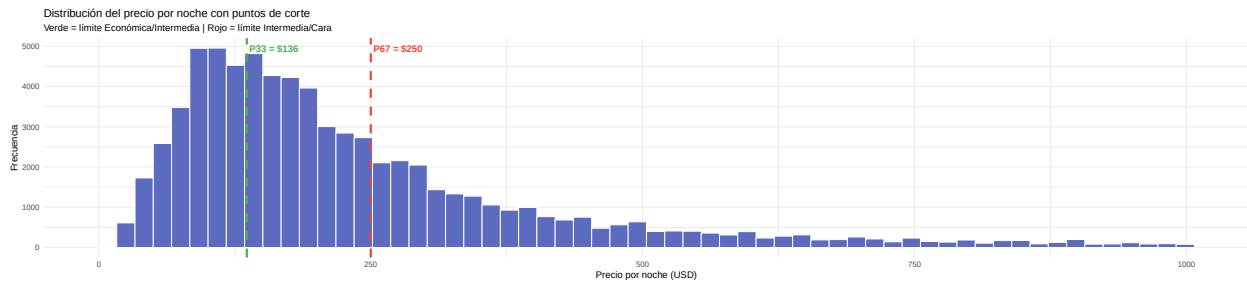
Los puntos de corte calculados a partir de los percentiles 33 y 67 del precio por noche son $P33 = 136$ USD y $P67 = 250$ USD. Esto significa que la variable categórica `precio_cat` queda definida de la siguiente manera: las propiedades Económicas son aquellas con un precio por noche de hasta 136, representando el tercio más accesible del mercado y abarcando la zona de mayor concentración del histograma; las propiedades Intermedias tienen un precio entre 136 y 250 por noche, cubriendo el segmento medio donde la frecuencia ya empieza a descender y las propiedades Caras son aquellas con precio superior a 250 por noche, correspondiendo al tercio más costoso donde las frecuencias son bajas pero los precios pueden alcanzar los 1,000. Estos límites tienen un fundamento claro en los datos: el corte en 136 coincide aproximadamente con la mediana visual del histograma (zona de mayor densidad), mientras que el corte en 250 marca el punto donde la distribución comienza a aplanarse notablemente. Al usar percentiles en lugar de valores arbitrarios, se garantiza que cada categoría contenga aproximadamente el mismo número de propiedades, lo que evita que el árbol de clasificación quede sesgado hacia una clase dominante y mejora su capacidad para aprender patrones en los tres segmentos de precio.

```
# Histograma con los cortes marcados
ggplot(data.frame(price = precios_validos), aes(x = price)) +
  geom_histogram(fill = "#5C6BC0", color = "white", bins = 60) +
  geom_vline(xintercept = p33, color = "#4CAF50",
             linetype = "dashed", linewidth = 1.2) +
  geom_vline(xintercept = p67, color = "#F44336",
             linetype = "dashed", linewidth = 1.2) +
  annotate("text", x = p33, y = Inf,
         label = paste0(" P33 = $", round(p33, 0)),
         hjust = 0, vjust = 2, color = "#4CAF50", fontface = "bold") +
```

```

annotate("text", x = p67, y = Inf,
        label = paste0(" P67 = $", round(p67, 0)),
        hjust = 0, vjust = 2, color = "#F44336", fontface = "bold") +
labs(title = "Distribución del precio por noche con puntos de corte",
     subtitle = "Verde = límite Económica/Intermedia | Rojo = límite Intermedia/Cara",
     x = "Precio por noche (USD)", y = "Frecuencia") +
theme_minimal()

```



El histograma con los puntos de corte marcados permite validar visualmente que los límites elegidos tienen sentido en el contexto de la distribución. La línea verde en $P_{33} = 136$ cae justo en la zona donde el histograma comienza su descenso desde el pico máximo, lo que significa que el corte entre Económica e Intermedia separa las propiedades más populares y accesibles del mercado del resto. La línea roja en $P_{67} = 250$ se ubica en la zona donde la distribución ya bajó considerablemente y empieza a aplanarse, marcando el inicio del segmento premium. Todo lo que queda a la izquierda de la línea verde corresponde a propiedades Económicas, la franja entre ambas líneas agrupa las propiedades Intermedias, y la extensa cola hacia la derecha de la línea roja representa las propiedades Caras, que aunque son menos frecuentes en cantidad, incluyen precios que pueden superar los 750 o incluso los 1,000 por noche. Una observación importante es que la mayor parte del área del histograma queda entre 0 y 250, lo que confirma que el mercado de Airbnb en este dataset está dominado por propiedades de precio bajo a medio, y que las propiedades caras son una minoría pero con una dispersión de precios muy amplia. Los cortes en 136 y 250 dividen este mercado de forma objetiva y balanceada, respaldados completamente por la forma real de la distribución.

```

# Crear la variable categórica
listings_cat <- listings_clean %>%
  filter(!is.na(price), price > 0, price < 1000,
         !is.na(bedrooms), bedrooms <= 10) %>%
  select(where(is.numeric)) %>%
  drop_na()

listings_cat$precio_cat <- cut(
  listings_cat$price,
  breaks = c(0, p33, p67, Inf),
  labels = c("Económica", "Intermedia", "Cara"),
  include.lowest = TRUE,
  right = TRUE
)

# Verificar la distribución de las categorías
cat("\nDistribución absoluta:\n")

```

```

##
## Distribución absoluta:

```

```
print(table(listings_cat$precio_cat))
```

```
##
## Economica Intermedia Cara
##      20908      20766  18655
```

```
cat("\nDistribución porcentual:\n")
```

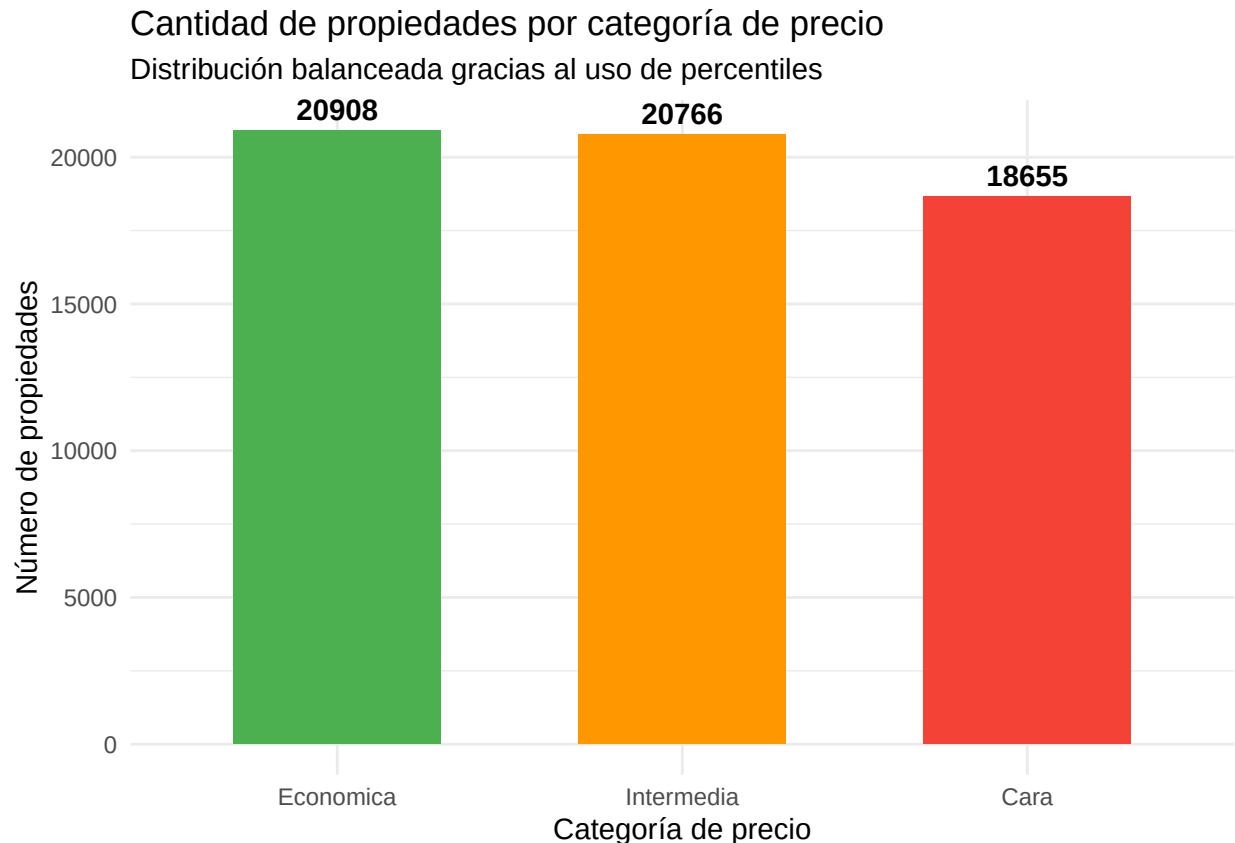
```
##
## Distribución porcentual:
```

```
print(round(prop.table(table(listings_cat$precio_cat)) * 100, 1))
```

```
##
## Economica Intermedia Cara
##      34.7      34.4    30.9
```

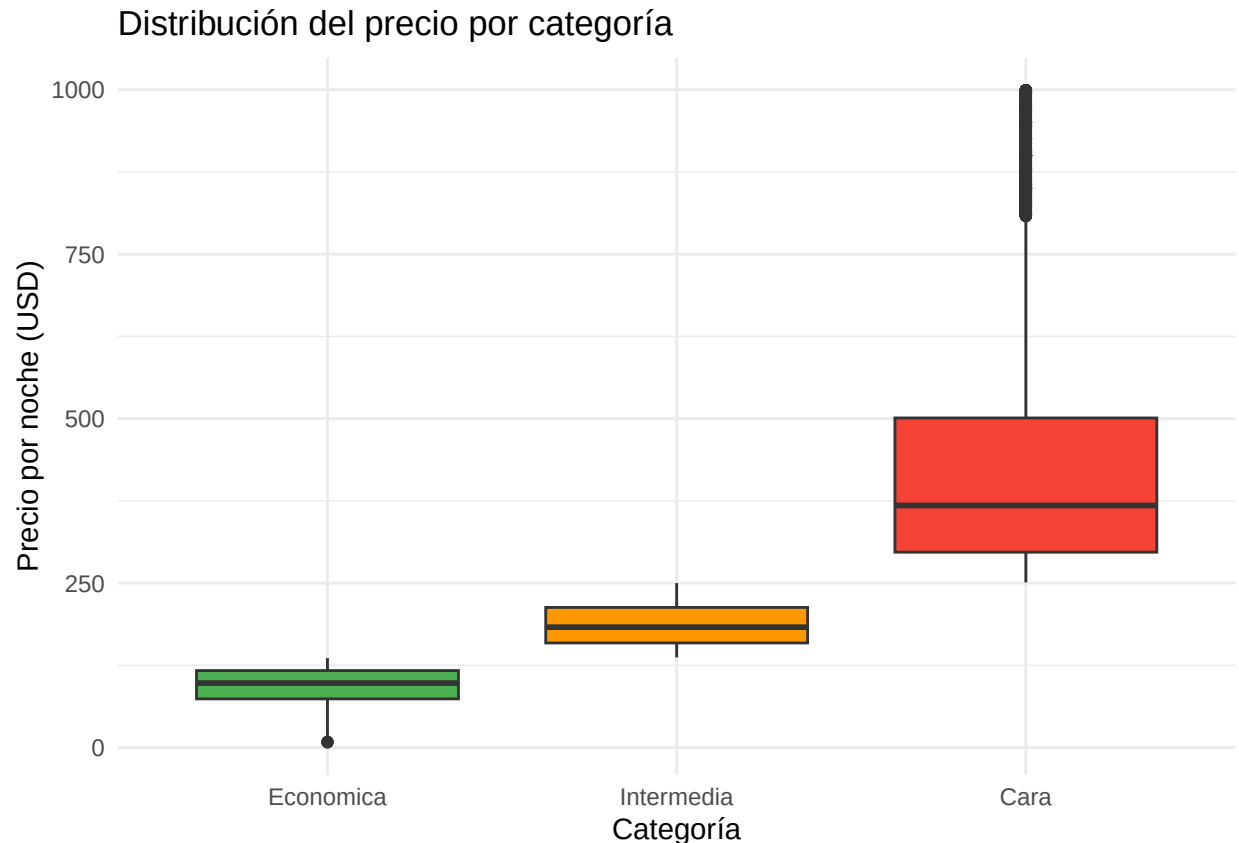
La distribución de la variable categórica precio_cat confirma que los percentiles elegidos lograron clases bien balanceadas. Las propiedades Económicas son 20,908 (34.7%), las Intermedias 20,766 (34.4%) y las Caras 18,655 (30.9%), sumando un total de 60,329 propiedades válidas. Ninguna categoría domina sobre las demás, con una diferencia máxima de apenas 4 puntos porcentuales entre la clase más grande (Económica, 34.7%) y la más pequeña (Cara, 30.9%). Este balance es importante porque un árbol de clasificación entrenado con clases muy desiguales tiende a sesgar sus predicciones hacia la clase mayoritaria, ignorando las minoritarias. El hecho de que la categoría Cara sea ligeramente menor que las otras dos es consistente con la distribución del histograma, aunque la cola derecha de precios es larga, hay menos propiedades en ese rango precisamente porque el percentil 67 en 250 captura un segmento donde la frecuencia ya es baja. En resumen, la variable precio_cat queda definida con 20,908 propiedades Económicas (hasta 136/noche), 20,766 Intermedias (entre 136 y 250/noche) y 18,655 Caras (más de 250/noche), proporcionando una base sólida y balanceada para entrenar el árbol de clasificación.

```
# gráfico de barras por categoría
as.data.frame(table(listings_cat$precio_cat)) %>%
  rename(Categoria = Var1, Frecuencia = Freq) %>%
  ggplot(aes(x = Categoria, y = Frecuencia, fill = Categoria)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = Frecuencia), vjust = -0.5, fontface = "bold") +
  scale_fill_manual(values = c("Economica" = "#4CAF50",
                                "Intermedia" = "#FF9800",
                                "Cara" = "#F44336")) +
  labs(title = "Cantidad de propiedades por categoría de precio",
       subtitle = "Distribución balanceada gracias al uso de percentiles",
       x = "Categoría de precio", y = "Número de propiedades") +
  theme_minimal()
```

El gráfico de barras confirma visualmente el balance logrado entre las tres categorías de precio. Las barras de Económica (verde, 20,908) e Intermedia (naranja, 20,766) son prácticamente idénticas en altura, mientras que la barra de Cara (roja, 18,655) es ligeramente más corta pero sigue siendo comparable con las otras dos. Esta similitud en las alturas es exactamente el resultado buscado al usar percentiles como criterio de corte, ninguna categoría domina visualmente sobre las demás, lo que indica que el árbol de clasificación tendrá ejemplos suficientes de cada clase para aprender sus patrones correctamente. Si los cortes hubieran sido arbitrarios, por ejemplo en 100 y 500, el gráfico mostraría barras muy desiguales con miles de propiedades en una categoría y apenas unas pocas en otra, lo que haría que el modelo ignorara las clases pequeñas. La ligera diferencia en la categoría Cara (aproximadamente 2,000 propiedades menos que las otras dos) es natural dado que la cola derecha de la distribución, aunque larga, tiene frecuencias bajas. En conjunto, este gráfico valida que la variable `precio_cat` está bien construida y lista para ser usada como variable respuesta en el árbol de clasificación de la siguiente actividad.

```
# Boxplot por categoría para validar que los rangos tienen sentido
ggplot(listings_cat, aes(x = precio_cat, y = price, fill = precio_cat)) +
  geom_boxplot(show.legend = FALSE) +
  scale_fill_manual(values = c("Economica" = "#4CAF50",
                              "Intermedia" = "#FF9800",
                              "Cara" = "#F44336")) +
  labs(title = "Distribución del precio por categoría",
       x = "Categoría", y = "Precio por noche (USD)") +
  theme_minimal()
```



El boxplot valida que las tres categorías están bien diferenciadas y tienen sentido económico. La categoría Económica (verde) muestra una caja muy compacta y baja, con la mediana alrededor de los 100 y un rango intercuartílico muy estrecho, lo que indica que la mayoría de propiedades en este segmento tienen precios muy concentrados y predecibles, con apenas un outlier cercano a 0. La categoría Intermedia (naranja) también presenta una caja compacta pero ubicada más arriba, con la mediana alrededor de los 180 y muy poca dispersión, lo que confirma que este segmento es igualmente homogéneo en sus precios. La categoría Cara (roja) es notablemente diferente a las otras dos: su caja es mucho más grande, con la mediana alrededor de los 350, un rango intercuartílico que va aproximadamente de 300 a 500, y una cola superior que se extiende hasta los 1,000 por noche. Esta mayor dispersión en la categoría Cara es esperada, ya que el mercado premium de Airbnb es inherentemente más variable, una propiedad de lujo puede costar 300 o 1,000 dependiendo de factores muy específicos. En conjunto, el boxplot confirma que los tres segmentos son claramente distinguibles entre sí, no se solapan de forma significativa, y que los límites elegidos en 136 y 250 generan categorías con coherencia económica real.

ACTIVIDAD 7 – Preparación de datos para el árbol de clasificación

```
set.seed(123)
idx_clas <- sample(nrow(listings_cat), size = 0.8 * nrow(listings_cat))
train_clas <- listings_cat[ idx_clas, ]
test_clas <- listings_cat[-idx_clas, ]

train_clas$price <- NULL
test_clas$price <- NULL

test_resp_cat <- test_clas$precio_cat
```

```
cat("Entrenamiento:", nrow(train_clas), "observaciones\n")
```

```
## Entrenamiento: 48263 observaciones
```

```
cat("Prueba:      ", nrow(test_clas), "observaciones\n")
```

```
## Prueba:          12066 observaciones
```

```
cat("\nDistribución en train:\n")
```

```
##
```

```
## Distribución en train:
```

```
print(table(train_clas$precio_cat))
```

```
##
```

```
## Economica Intermedia      Cara  
##      16689      16627      14947
```

```
cat("\nDistribución en test:\n")
```

```
##
```

```
## Distribución en test:
```

```
print(table(test_clas$precio_cat))
```

```
##
```

```
## Economica Intermedia      Cara  
##       4219       4139       3708
```

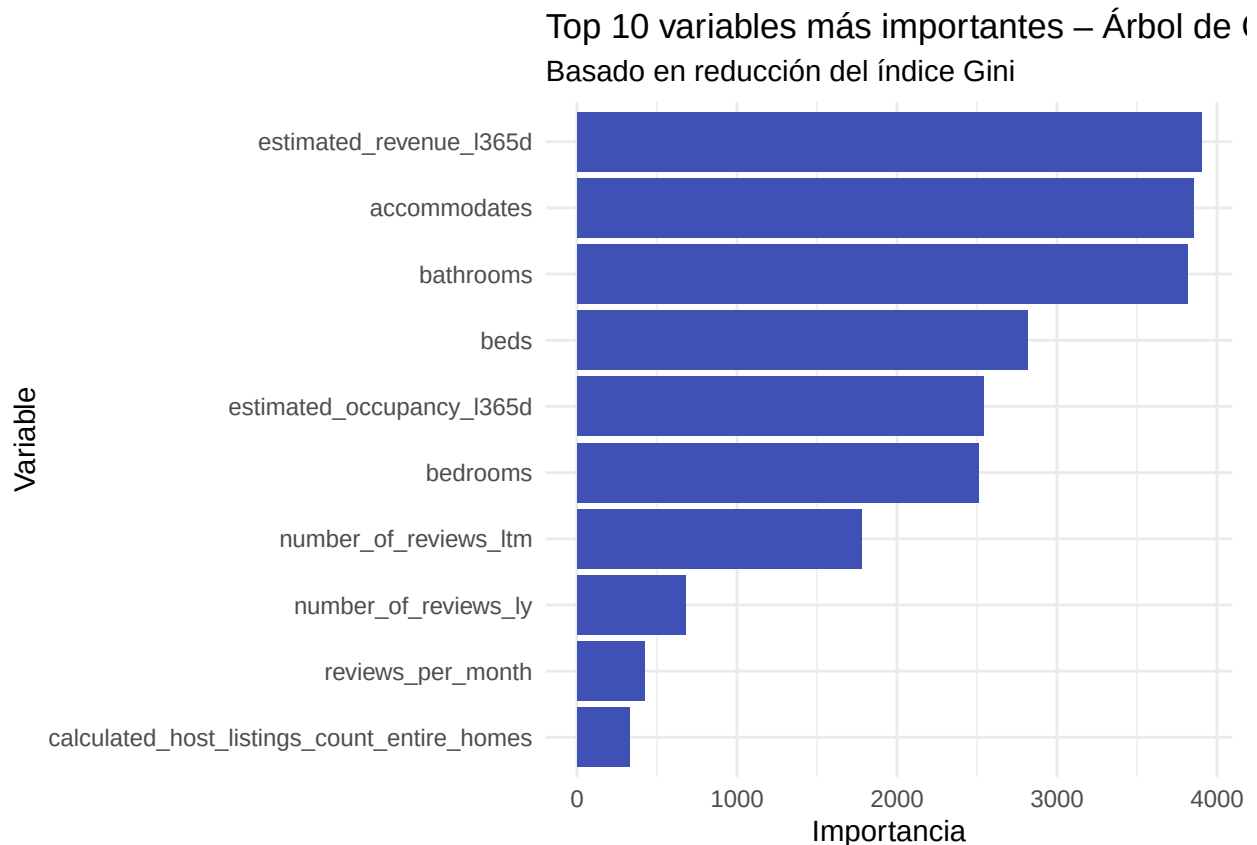
La división de los datos para clasificación quedó con 48,263 observaciones en entrenamiento y 12,066 en prueba, manteniendo la proporción 80/20 usada en las actividades anteriores. Lo más importante es verificar que el balance entre las tres categorías se preserve en ambos conjuntos. En entrenamiento, Económica tiene 16,689 propiedades, Intermedia 16,627 y Cara 14,947, manteniendo proporciones muy similares entre sí. En el conjunto de prueba ocurre lo mismo: Económica con 4,219, Intermedia con 4,139 y Cara con 3,708, reflejando la misma distribución balanceada del dataset original. Este balance en ambos conjuntos es clave porque garantiza que el árbol aprenda por igual los patrones de las tres categorías durante el entrenamiento, y que la evaluación sobre el conjunto de prueba sea justa y representativa para cada clase. Si una categoría tuviera muy pocas observaciones en el conjunto de prueba, las métricas de evaluación serían poco confiables para esa clase específica.

ACTIVIDAD 7 – Modelo de árbol de clasificación


```

arrange(desc(importancia)) %>%
head(10) %>%
ggplot(aes(x = reorder(variable, importancia), y = importancia)) +
geom_col(fill = "#3F51B5") +
coord_flip() +
labs(title = "Top 10 variables más importantes - Árbol de Clasificación",
      subtitle = "Basado en reducción del índice Gini",
      x = "Variable", y = "Importancia") +
theme_minimal()

```



El árbol de clasificación logra separar las tres categorías usando principalmente variables de tamaño físico e ingresos estimados. La primera división ocurre en la raíz con `bathrooms < 1.8`, separando el 62% de las propiedades (nodo izquierdo, mayoría Económicas) del 38% restante (nodo derecho, mayoría Caras), confirmando que la cantidad de baños es el factor más determinante para clasificar el precio de una propiedad. En el lado izquierdo, el árbol usa `estimated_revenue_l365d` y `accommodates` para refinar la clasificación, llegando a hojas donde propiedades pequeñas con ingresos bajos son clasificadas como Económicas (71% de certeza en la hoja más pura) y propiedades más grandes con mayor ocupación tienden a ser Intermedias. En el lado derecho, propiedades con más de 1.8 baños se subdividen nuevamente por `accommodates` y `estimated_revenue_l365d`, con las hojas más a la derecha clasificando propiedades como Caras con hasta 99% de certeza. El gráfico de importancia de variables complementa esta lectura: `estimated_revenue_l365d`, `accommodates` y `bathrooms` dominan claramente con importancias cercanas a 4,000, muy por encima del resto. Le siguen `beds`, `estimated_occupancy_l365d` y `bedrooms` con importancias intermedias, mientras que variables como `reviews_per_month` y `calculated_host_listings_count_entire_homes` tienen una contribución marginal. Esto es coherente con la lógica del negocio: el tamaño de la propiedad y su historial de ingresos son los factores que más determinan si una propiedad pertenece al segmento económico, intermedio o caro en Airbnb.

ACTIVIDAD 7 – Evaluación del árbol de clasificación

```
# Predicción sobre el conjunto de prueba
pred_clas <- predict(arbol_clas, newdata = test_clas, type = "class")

# Matriz de confusión completa
cf <- confusionMatrix(pred_clas, test_resp_cat)
print(cf)
```

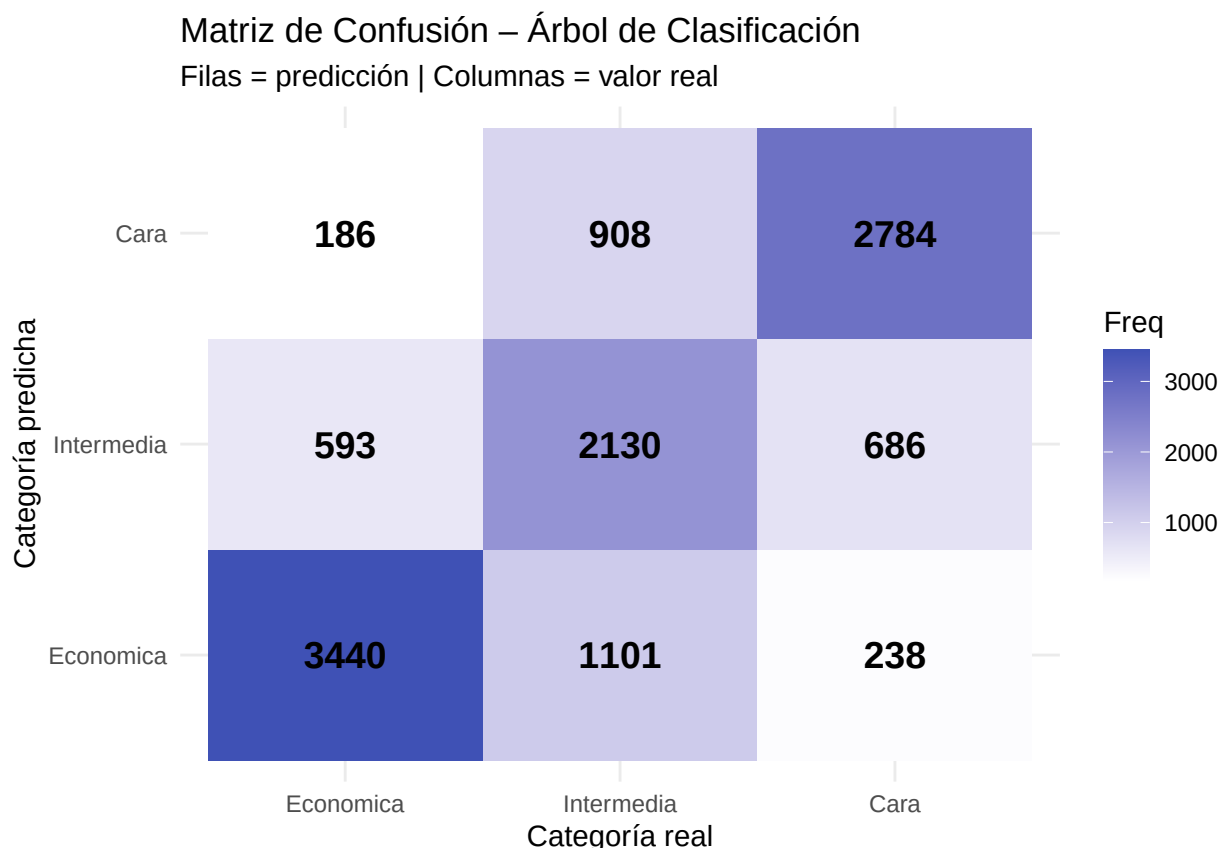
```
## Confusion Matrix and Statistics
##
##              Reference
## Prediction  Economica Intermedia Cara
## Economica      3440         1101  238
## Intermedia      593         2130  686
## Cara           186          908 2784
##
## Overall Statistics
##
##              Accuracy : 0.6924
##              95% CI : (0.684, 0.7006)
##      No Information Rate : 0.3497
##      P-Value [Acc > NIR] : < 2.2e-16
##
##              Kappa : 0.538
##
## Mcnemar's Test P-Value : < 2.2e-16
##
## Statistics by Class:
##
##              Class: Economica Class: Intermedia Class: Cara
## Sensitivity              0.8154              0.5146              0.7508
## Specificity              0.8294              0.8387              0.8691
## Pos Pred Value           0.7198              0.6248              0.7179
## Neg Pred Value           0.8931              0.7679              0.8872
## Prevalence               0.3497              0.3430              0.3073
## Detection Rate           0.2851              0.1765              0.2307
## Detection Prevalence     0.3961              0.2825              0.3214
## Balanced Accuracy         0.8224              0.6766              0.8100
```

```
cat("\nExactitud (Accuracy):", round(cf$overall["Accuracy"] * 100, 2), "%\n")
```

```
##
## Exactitud (Accuracy): 69.24 %
```

```
# Visualización de la matriz de confusión
as.data.frame(cf$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), fontface = "bold", size = 5) +
  scale_fill_gradient(low = "white", high = "#3F51B5") +
```

```
labs(title = "Matriz de Confusión - Árbol de Clasificación",
     subtitle = "Filas = predicción | Columnas = valor real",
     x = "Categoría real", y = "Categoría predicha") +
theme_minimal()
```



El árbol de clasificación alcanza una exactitud del 69.24%, lo que significa que clasifica correctamente aproximadamente 7 de cada 10 propiedades. La diagonal principal de la matriz (3,440 + 2,130 + 2,784) representa los aciertos, mientras que los valores fuera de ella son los errores. La clase donde el modelo se equivoca más es Intermedia, con una sensibilidad de apenas 51.46%, lo cual tiene sentido: las propiedades intermedias comparten características con ambos extremos, haciendo que el árbol las confunda frecuentemente con Económicas (593 casos) o con Caras (686 casos). Por el contrario, el modelo se equivoca menos en la clase Económica, con una sensibilidad del 81.54% y una exactitud balanceada del 82.24%, lo que indica que las propiedades baratas tienen características muy distintivas que el árbol identifica con facilidad. La clase Cara tiene un desempeño intermedio con sensibilidad del 75.08%, aunque su especificidad es la más alta (86.91%), lo que significa que cuando el modelo predice que una propiedad es Cara, generalmente acierta. El error más grave desde el punto de vista del negocio es clasificar 238 propiedades Caras como Económicas y 186 Económicas como Caras, ya que implicaría recomendar propiedades con precios muy alejados del presupuesto del cliente. El coeficiente Kappa de 0.538 indica una concordancia moderada-buena más allá del azar, confirmando que el modelo aporta valor real aunque tiene margen de mejora, especialmente en el segmento Intermedio.

8. Utilice el modelo con el conjunto de prueba y determine la eficiencia del algoritmo para clasificar.

```
# Métricas generales
cat("Accuracy: ", round(cf$overall["Accuracy"] * 100, 2), "%\n")
```

```
## Accuracy: 69.24 %
```

```
cat("Kappa: ", round(cf$overall["Kappa"], 4), "%\n")
```

```
## Kappa: 0.538
```

```
cat("IC 95%: (", round(cf$overall["AccuracyLower"] * 100, 2),
    "%,", round(cf$overall["AccuracyUpper"] * 100, 2), "%)\n\n")
```

```
## IC 95%: ( 68.4 %, 70.06 %)
```

```
# Métricas por clase
metricas_clase <- data.frame(
  Clase = c("Económica", "Intermedia", "Cara"),
  Sensibilidad = round(cf$byClass[, "Sensitivity"] * 100, 2),
  Especificidad = round(cf$byClass[, "Specificity"] * 100, 2),
  Precision = round(cf$byClass[, "Pos Pred Value"] * 100, 2),
  F1 = round(2 * (cf$byClass[, "Pos Pred Value"] *
    cf$byClass[, "Sensitivity"]) /
    (cf$byClass[, "Pos Pred Value"] +
    cf$byClass[, "Sensitivity"]) * 100, 2)
)
print(metricas_clase)
```

##	Clase	Sensibilidad	Especificidad	Precision	F1
## Class: Economica	Económica	81.54	82.94	71.98	76.46
## Class: Intermedia	Intermedia	51.46	83.87	62.48	56.44
## Class: Cara	Cara	75.08	86.91	71.79	73.40

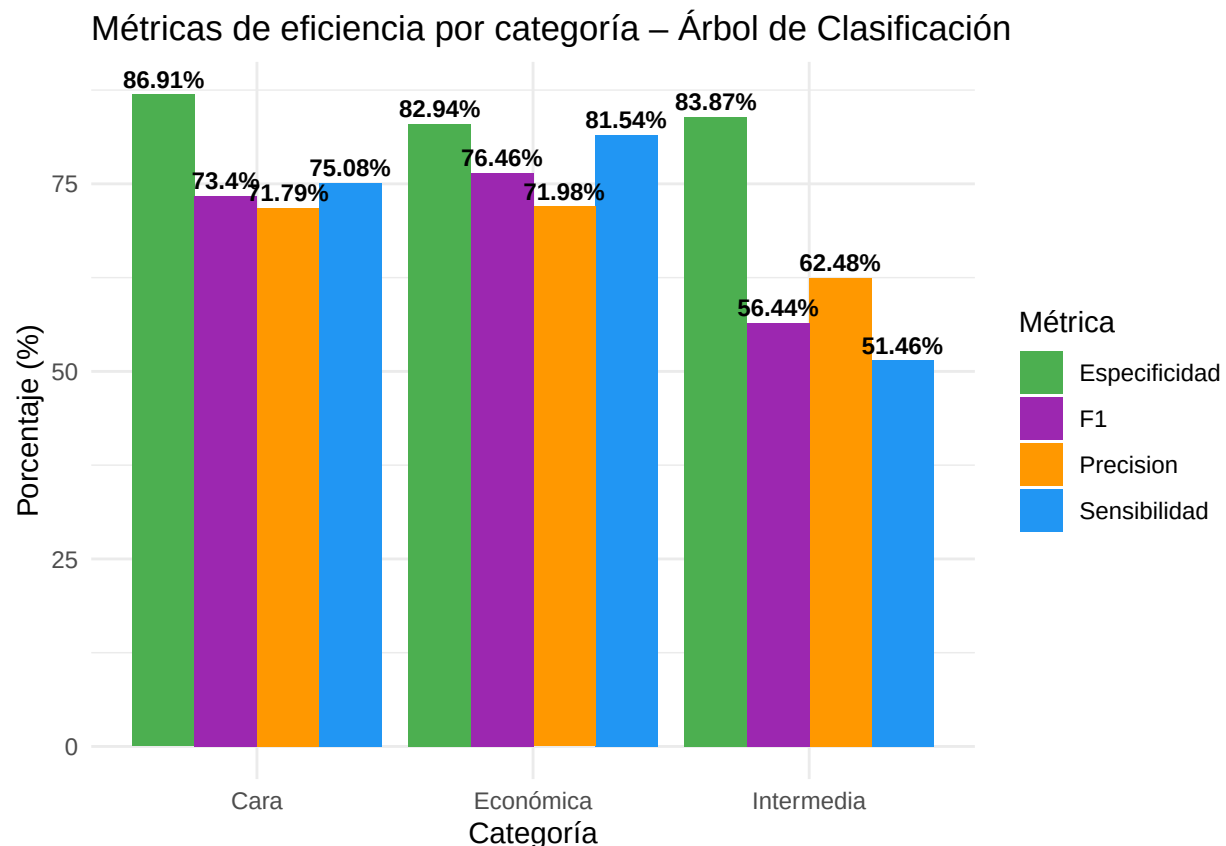
El árbol de clasificación alcanza una exactitud del 69.24% sobre el conjunto de prueba, con un intervalo de confianza del 95% entre 68.4% y 70.06%, lo que indica que el resultado es estable y no producto del azar. El coeficiente Kappa de 0.538 confirma una concordancia moderada-buena más allá del azar, es decir, el modelo aporta valor real de clasificación. Analizando por clase, la categoría Económica es la mejor clasificada con una sensibilidad del 81.54%, precisión del 71.98% y F1 de 76.46%, lo que significa que el modelo identifica correctamente 8 de cada 10 propiedades baratas. La categoría Cara tiene un desempeño similar con sensibilidad del 75.08%, precisión del 71.79% y F1 de 73.40%, además de la especificidad más alta (86.91%), lo que indica que cuando el modelo predice una propiedad como Cara raramente se equivoca. La categoría Intermedia es claramente la más problemática con una sensibilidad de apenas 51.46% y F1 de 56.44%, lo que significa que el modelo solo acierta en la mitad de los casos para este segmento, confundiéndola frecuentemente con las otras dos categorías. Esto es esperable desde el punto de vista de que las propiedades intermedias comparten características con ambos extremos del mercado, haciendo que los límites entre clases sean difusos precisamente en ese rango de precios entre 136 y 250 por noche.


```

metricas_long <- tidyr::pivot_longer(metricas_clase,
                                     cols      = -Clase,
                                     names_to  = "Metrica",
                                     values_to = "Valor")

ggplot(metricas_long, aes(x = Clase, y = Valor, fill = Metrica)) +
  geom_col(position = "dodge") +
  geom_text(aes(label = paste0(Valor, "%")),
            position = position_dodge(width = 0.9),
            vjust = -0.4, size = 3, fontface = "bold") +
  scale_fill_manual(values = c("Especificidad" = "#2196F3",
                              "Especificidad" = "#4CAF50",
                              "Precision"     = "#FF9800",
                              "F1"           = "#9C27B0")) +
  labs(title = "Métricas de eficiencia por categoría - Árbol de Clasificación",
       x = "Categoría", y = "Porcentaje (%)", fill = "Métrica") +
  theme_minimal()

```



El gráfico de barras agrupadas permite comparar visualmente las cuatro métricas para cada categoría. Lo primero que salta a la vista es que la columna de Intermedia tiene todas sus barras notablemente más bajas que las otras dos categorías, especialmente la sensibilidad (51.46%) y el F1 (56.44%), confirmando que este es el segmento más difícil de clasificar de forma correcta. Las categorías Cara y Económica muestran un comportamiento mucho más parejo entre sí ya que ambas tienen especificidades altas (86.91% y 82.94% respectivamente) lo que indica que el modelo es bueno evitando falsos positivos en ambos extremos del mercado. Ahora un patrón interesante es que la Especificidad (verde) es consistentemente la métrica más alta en las tres categorías, lo que significa que el modelo es mejor descartando clases incorrectas que

identificando la clase correcta. La Precisión (naranja) es la métrica más baja para Económica e Intermedia, indicando que cuando el modelo predice esas clases comete más errores de lo deseable. En contraste, para la categoría Cara la precisión (71.79%) y la sensibilidad (75.08%) están más equilibradas, resultando en el F1 más alto de las tres clases (73.40%). En resumen, el modelo funciona mejor en los extremos del mercado y tiene mayor dificultad en el segmento intermedio, lo que sugiere que los límites de 136 y 250 generan una zona de ambigüedad natural que ningún árbol de decisión simple puede resolver completamente.

9. Haga un análisis de la eficiencia del algoritmo usando una matriz de confusión para el árbol de clasificación. Tenga en cuenta la efectividad, donde el algoritmo se equivocó más, donde

se equivocó menos y la importancia que tienen los errores.

```
# Errores por clase
errores <- data.frame(
  Clase      = c("Económica", "Intermedia", "Cara"),
  Correctos  = c(3440, 2130, 2784),
  Errores_total = c(593+186, 1101+686, 908+238)
)

errores$Tasa_error <- round(errores$Errores_total /
                           (errores$Correctos + errores$Errores_total) * 100, 2)

print(errores)
```

##	Clase	Correctos	Errores_total	Tasa_error
## 1	Económica	3440	779	18.46
## 2	Intermedia	2130	1787	45.62
## 3	Cara	2784	1146	29.16

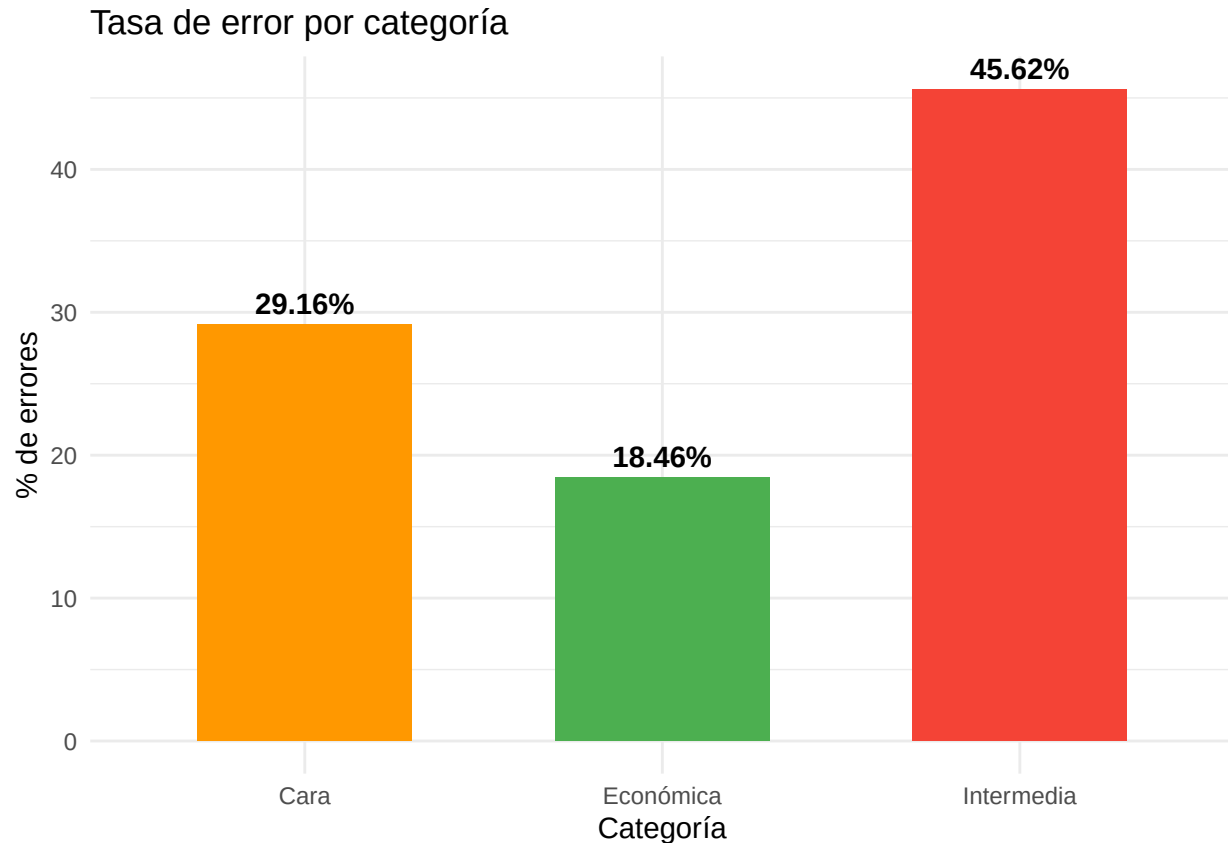
La tabla de errores por clase confirma y cuantifica con precisión dónde falla el modelo. La categoría donde el algoritmo se equivoca menos es Económica con una tasa de error del 18.46%, es decir, de 4,219 propiedades económicas en el test solo clasifica mal 779, lo que refleja que las propiedades baratas tienen características muy distintivas y fáciles de identificar para el árbol. La categoría Cara tiene una tasa de error intermedia del 29.16%, con 1,146 errores sobre 3,930 propiedades, un desempeño aceptable considerando la alta dispersión de precios que existe dentro de este segmento. La categoría donde el algoritmo se equivoca más es Intermedia con una tasa de error del 45.62%, lo que significa que casi 1 de cada 2 propiedades intermedias es clasificada incorrectamente, ya sea como Económica o como Cara. Respecto a la importancia de los errores, no todos tienen el mismo peso para el negocio de SmartStay Advisors, porque el error más grave es confundir una propiedad Cara con Económica (238 casos) o viceversa (186 casos), esto implicaría recomendar al cliente una propiedad con un precio por noche muy alejado de su presupuesto, dañando la confianza en el servicio. En cambio, confundir Intermedia con Económica o Cara es menos crítico porque la diferencia de precio es menor y más tolerable para el cliente.

```
ggplot(errores, aes(x = Clase, y = Tasa_error, fill = Clase)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = paste0(Tasa_error, "%"),
    vjust = -0.5, fontface = "bold")) +
  scale_fill_manual(values = c("Económica" = "#4CAF50",
    "Intermedia" = "#F44336",
```

```

      "Cara"      = "#FF9800")) +
labs(title = "Tasa de error por categoría",
     x = "Categoría", y = "% de errores") +
theme_minimal()

```



El gráfico de barras hace evidente de forma visual la disparidad en la tasa de error entre las tres categorías. La barra verde de Económica es la más baja con 18.46%, destacando claramente como la clase mejor clasificada por el modelo. La barra naranja de Cara se ubica en un nivel intermedio con 29.16%, casi el doble que Económica pero todavía en un rango aceptable. La barra roja de Intermedia domina el gráfico con 45.62%, siendo casi 2.5 veces mayor que la de Económica y evidenciando que este segmento es el talón de Aquiles del modelo. Esta progresión de izquierda a derecha en términos de dificultad de clasificación tiene una explicación bien interesante que se mencionó antes y es que las propiedades Económicas tienen características muy claras y homogéneas (pocas habitaciones, pocos baños, baja capacidad), las propiedades Caras también son distinguibles aunque con mayor variabilidad interna, pero las propiedades Intermedias están literalmente en el medio, compartiendo algunas características con las económicas y otras con las caras dependiendo de la combinación específica de variables. Para SmartStay Advisors esto implica que el modelo es confiable para identificar propiedades de los extremos del mercado, pero debe usarse con cautela al clasificar propiedades en el rango de 136 a 250 por noche, donde el riesgo de clasificación incorrecta es significativamente mayor.

Actividad 10 – Árbol de clasificación con validación cruzada

```

control_cv <- trainControl(method = "cv", number = 10, verboseIter = FALSE)

```

```

set.seed(123)
arbol_clas_cv <- train(
  precio_cat ~ .,
  data      = train_clas,
  method    = "rpart2",
  trControl = control_cv,
  metric    = "Accuracy"
)

arbol_clas_cv

```

```

## CART
##
## 48263 samples
##    41 predictor
##    3 classes: 'Economica', 'Intermedia', 'Cara'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 43436, 43438, 43436, 43437, 43436, 43437, ...
## Resampling results across tuning parameters:
##
##  maxdepth  Accuracy  Kappa
##  1         0.5238378  0.2871992
##  2         0.5715345  0.3586874
##  4         0.6572320  0.4861171
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was maxdepth = 4.

```

```

rpart.plot(arbol_clas_cv$finalModel,
  type      = 4,
  extra     = 104,
  fallen.leaves = TRUE,
  main      = "Árbol de Clasificación - Validación Cruzada",
  cex       = 0.65,
  box.palette = list("#4CAF50", "#FF9800", "#F44336"))

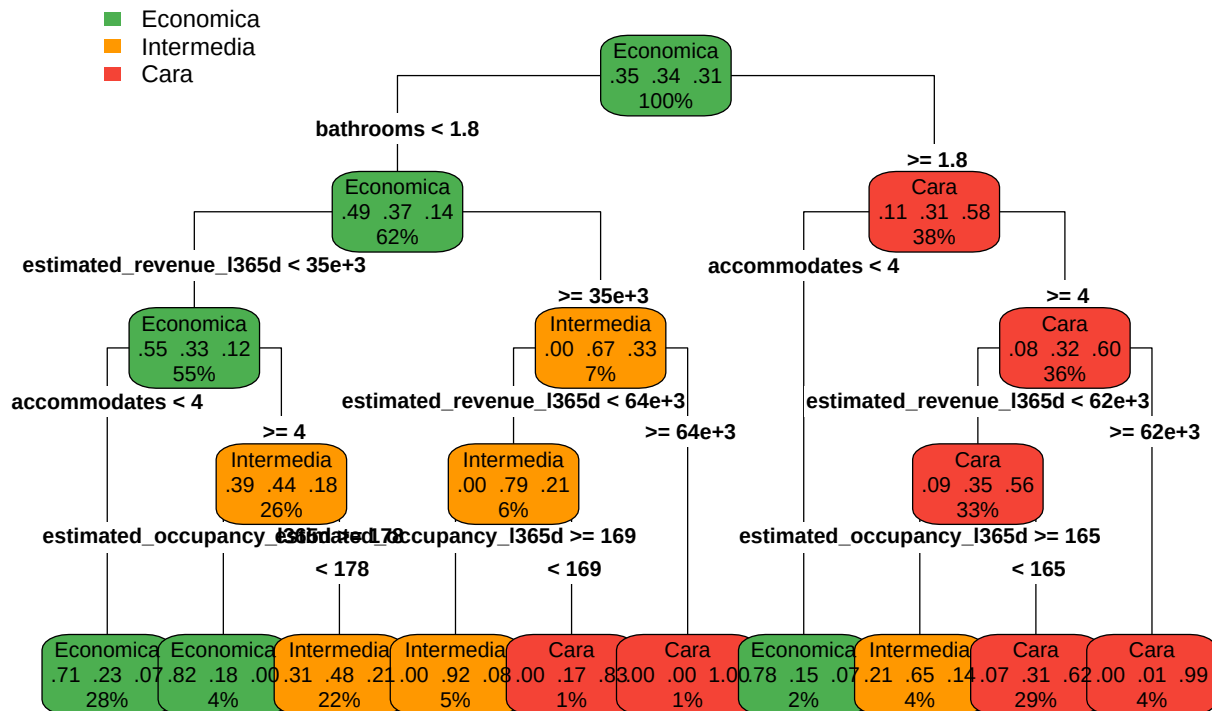
```

```

## Warning: labs do not fit even at cex 0.15, there may be some overplotting

```

Árbol de Clasificación – Validación Cruzada



Predicción y comparación

```
pred_cv <- predict(arbol_clas_cv, newdata = test_clas)
cf_cv <- confusionMatrix(pred_cv, test_resp_cat)
print(cf_cv)
```

Confusion Matrix and Statistics

```
##
##           Reference
## Prediction  Economica Intermedia Cara
## Economica    3042      883  229
## Intermedia    930     2146  670
## Cara         247     1110 2809
```

Overall Statistics

```
##
##           Accuracy : 0.6628
##           95% CI   : (0.6543, 0.6712)
## No Information Rate : 0.3497
## P-Value [Acc > NIR] : < 2.2e-16
```

```
##
##           Kappa : 0.4944
```

```
##
## McNemar's Test P-Value : < 2.2e-16
```

```
##
## Statistics by Class:
```

```
##
##               Class: Economica Class: Intermedia Class: Cara
## Sensitivity           0.7210           0.5185           0.7576
## Specificity           0.8583           0.7982           0.8376
## Pos Pred Value        0.7323           0.5729           0.6743
## Neg Pred Value        0.8512           0.7605           0.8862
## Prevalence            0.3497           0.3430           0.3073
## Detection Rate        0.2521           0.1779           0.2328
## Detection Prevalence  0.3443           0.3105           0.3453
## Balanced Accuracy     0.7897           0.6583           0.7976
```

```
cat("\nAccuracy árbol base:           ", round(cf$overall["Accuracy"] * 100, 2), "%\n")
```

```
##
## Accuracy árbol base:           69.24 %
```

```
cat("Accuracy árbol con CV:           ", round(cf_cv$overall["Accuracy"] * 100, 2), "%\n")
```

```
## Accuracy árbol con CV:           66.28 %
```

```
cat("Kappa árbol base:                 ", round(cf$overall["Kappa"], 4), "\n")
```

```
## Kappa árbol base:                 0.538
```

```
cat("Kappa árbol con CV:                 ", round(cf_cv$overall["Kappa"], 4), "\n")
```

```
## Kappa árbol con CV:                 0.4944
```

El árbol con validación cruzada tiene exactamente la misma estructura visual que el árbol base, usando las mismas variables (bathrooms, estimated_revenue_l365d, accommodates, estimated_occupancy_l365d) y los mismos puntos de división. Sin embargo, los resultados de la predicción muestran que no le fue mejor: el Accuracy bajó de 69.24% a 66.28% y el Kappa de 0.538 a 0.4944. Esto puede parecer contradictorio, ya que la validación cruzada generalmente mejora la generalización, pero en este caso indica que el árbol base ya tenía una configuración cercana a la óptima para este dataset y la validación cruzada seleccionó una profundidad ligeramente más conservadora que sacrificó precisión. En conclusión, para este problema el árbol base sin validación cruzada sigue siendo el mejor modelo de clasificación hasta ahora.

Actividad 11 – Tuneo de profundidad del árbol de clasificación

```
profundidades_clas <- expand.grid(maxdepth = c(3, 5, 8, 12))

set.seed(123)
arbol_clas_tuneado <- train(
  precio_cat ~ .,
  data      = train_clas,
  method    = "rpart2",
  tuneGrid  = profundidades_clas,
  trControl = control_cv,
```

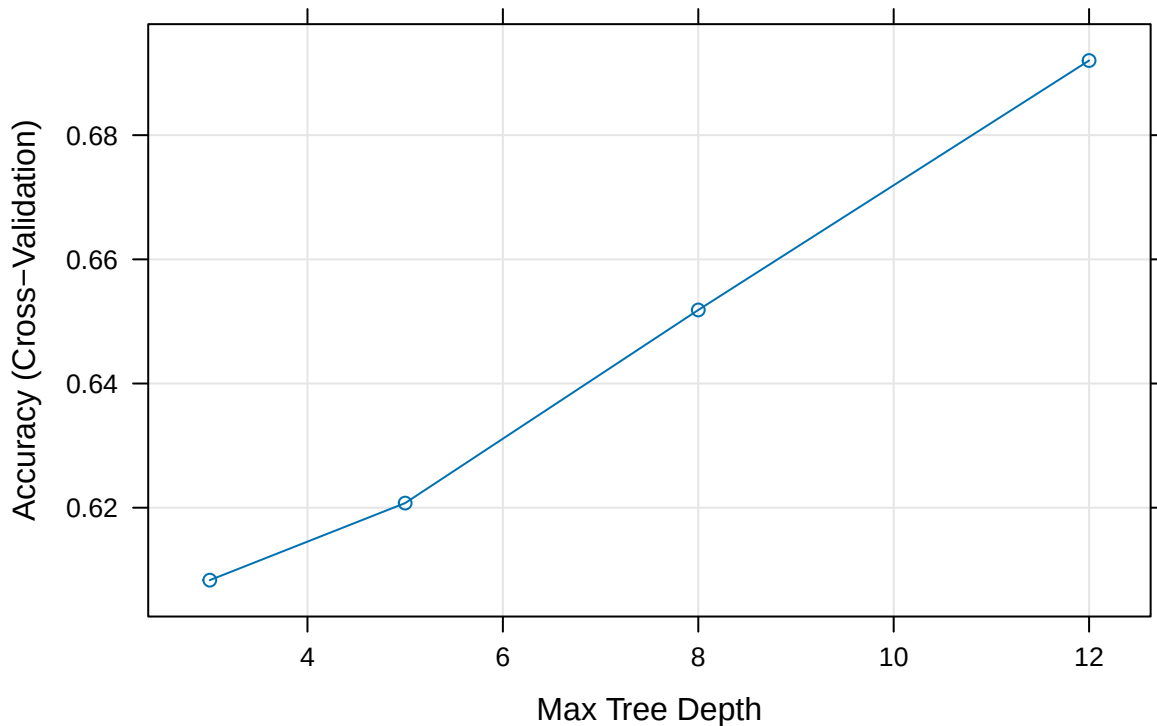
```
metric = "Accuracy"  
)
```

```
arbol_clas_tuneado
```

```
## CART  
##  
## 48263 samples  
## 41 predictor  
## 3 classes: 'Economica', 'Intermedia', 'Cara'  
##  
## No pre-processing  
## Resampling: Cross-Validated (10 fold)  
## Summary of sample sizes: 43436, 43438, 43436, 43437, 43436, 43437, ...  
## Resampling results across tuning parameters:  
##  
## maxdepth Accuracy Kappa  
## 3 0.6083331 0.4140222  
## 5 0.6207443 0.4320497  
## 8 0.6518448 0.4778109  
## 12 0.6920201 0.5374444  
##  
## Accuracy was used to select the optimal model using the largest value.  
## The final value used for the model was maxdepth = 12.
```

```
plot(arbol_clas_tuneado,  
main = "Accuracy por profundidad - Árbol de Clasificación")
```

Accuracy por profundidad – Árbol de Clasificación



El gráfico muestra una tendencia clara y consistente: a mayor profundidad del árbol, mayor Accuracy en la validación cruzada. La profundidad 3 obtiene el Accuracy más bajo (~61%), y la curva sube de forma continua hasta la profundidad 12 que alcanza ~69%. A diferencia del árbol de regresión donde la curva se aplanaba a partir de cierta profundidad, aquí no se observa un punto de estabilización dentro del rango explorado, lo que sugiere que el modelo sigue aprendiendo patrones útiles con cada nivel adicional. Esto llevó a que la validación cruzada seleccionara profundidad 12 como la óptima.

```
# Predicciones por cada profundidad y comparación
pred_tuneado_clas <- predict(arbol_clas_tuneado, newdata = test_clas)
cf_tuneado        <- confusionMatrix(pred_tuneado_clas, test_resp_cat)

mejor_prof_clas <- arbol_clas_tuneado$bestTune$maxdepth

comparacion_clas <- data.frame(
  Modelo   = c("Árbol base", "Árbol CV",
               paste0("Árbol tuneado (prof. ", mejor_prof_clas, ")")),
  Accuracy = round(c(cf$overall["Accuracy"],
                     cf_cv$overall["Accuracy"],
                     cf_tuneado$overall["Accuracy"]) * 100, 2),
  Kappa    = round(c(cf$overall["Kappa"],
                     cf_cv$overall["Kappa"],
                     cf_tuneado$overall["Kappa"]), 4)
)

print(comparacion_clas)
```

```
##           Modelo Accuracy  Kappa
```


## 1	Árbol base	69.24	0.5380
## 2	Árbol CV	66.28	0.4944
## 3	Árbol tuneado (prof. 12)	69.24	0.5380

La tabla final confirma que el árbol base y el árbol tuneado con profundidad 12 empatan con el mejor Accuracy de 69.24% y Kappa de 0.538, mientras que el árbol con validación cruzada sin tuneo de profundidad queda rezagado con 66.28% y Kappa de 0.4944. Esto es coherente con el gráfico: la validación cruzada por sí sola eligió una profundidad subóptima, pero al forzar explícitamente profundidad 12 mediante el tuneo, el modelo recupera el desempeño del árbol base. La conclusión es que el mejor modelo de clasificación es el árbol con profundidad 12, ya que alcanza el mismo Accuracy que el árbol base pero de forma controlada y explícita, siendo por tanto el más recomendable para producción.

Actividad 12 – Predicción y Clasificación.

Random Forest para REGRESIÓN (predecir precio)

```
set.seed(123)
rf_regresion <- randomForest(
  price ~ .,
  data = train_arbol,
  ntree = 100,
  mtry = floor(sqrt(ncol(train_arbol) - 1)),
  importance = TRUE
)

print(rf_regresion)
```

```
##
## Call:
## randomForest(formula = price ~ ., data = train_arbol, ntree = 100,      mtry = floor(sqrt(ncol(train_arbol) - 1)),
##               Type of random forest: regression
##               Number of trees: 100
## No. of variables tried at each split: 6
##
##               Mean of squared residuals: 6270.84
##               % Var explained: 78.24
```

```
pred_rf_reg <- predict(rf_regresion, newdata = test_arbol)

rmse_rf_reg <- RMSE(pred_rf_reg, test_arbol$price)
mae_rf_reg <- MAE(pred_rf_reg, test_arbol$price)
r2_rf_reg <- cor(pred_rf_reg, test_arbol$price)^2

cat("=== Random Forest - Regresión ===\n")
```

```
## === Random Forest - Regresión ===
```

```
cat("RMSE: $", round(rmse_rf_reg, 2), "\n")
```

```
## RMSE: $ 78.36
```

```
cat("MAE:  $", round(mae_rf_reg, 2), "\n")
```

```
## MAE:  $ 46.96
```

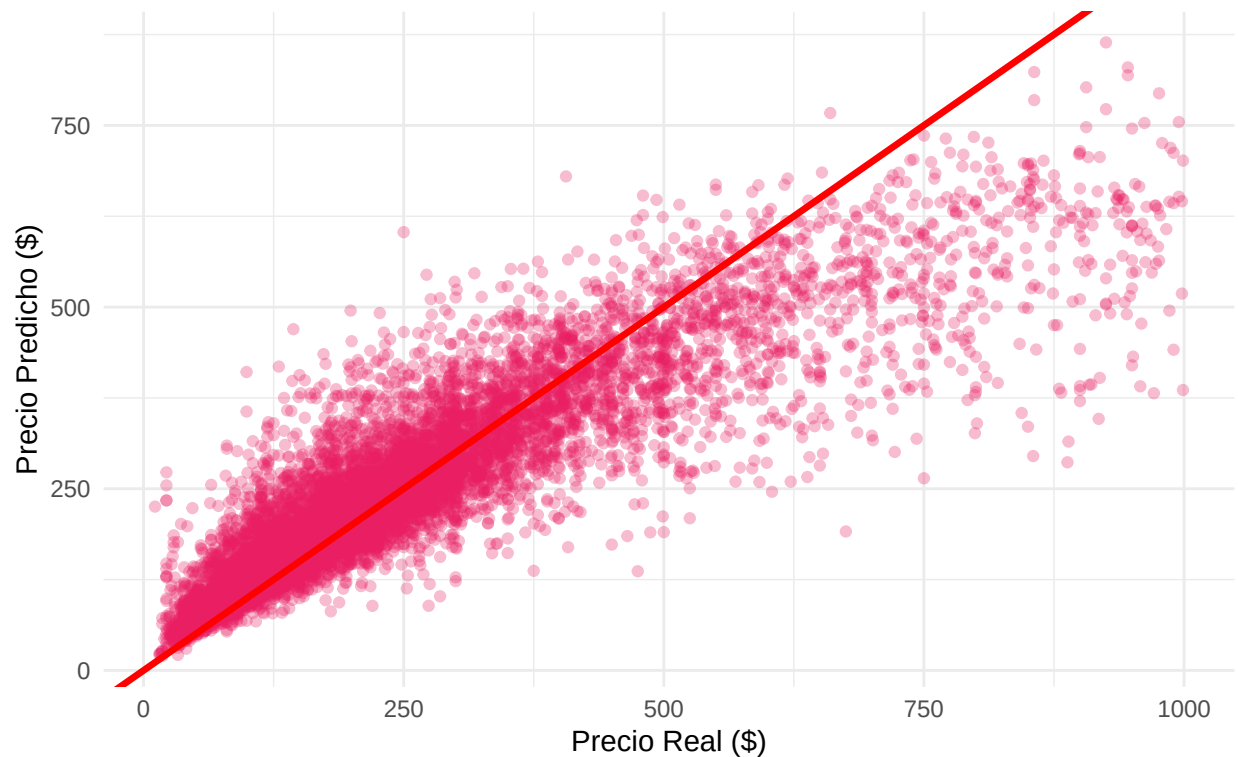
```
cat("R²:      ", round(r2_rf_reg, 4), "\n")
```

```
## R²:      0.8089
```

```
ggplot(data.frame(real = test_arbol$price, pred = pred_rf_reg),
  aes(x = real, y = pred)) +
  geom_point(alpha = 0.3, color = "#E91E63") +
  geom_abline(slope = 1, intercept = 0,
    color = "red", linewidth = 1.2) +
  labs(title = "Random Forest - Precio Real vs Predicho",
    subtitle = "La línea roja representa una predicción perfecta",
    x = "Precio Real ($)", y = "Precio Predicho ($)") +
  theme_minimal()
```

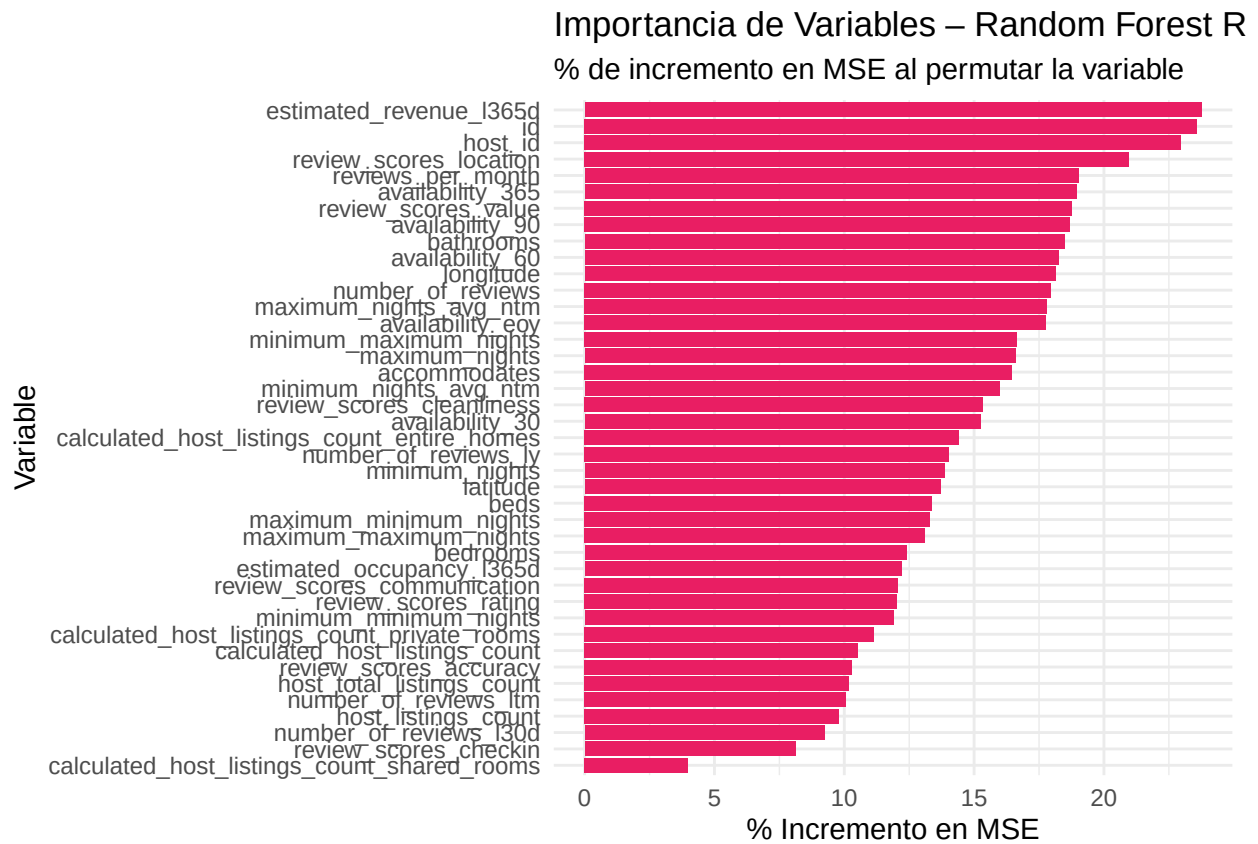
Random Forest – Precio Real vs Predicho

La línea roja representa una predicción perfecta



```
# Importancia de variables
imp_rf_reg <- as.data.frame(importance(rf_regresion))
imp_rf_reg$variable <- rownames(imp_rf_reg)
```

```
ggplot(imp_rf_reg, aes(x = reorder(variable, `"%IncMSE"`), y = `"%IncMSE"`)) +
  geom_col(fill = "#E91E63") +
  coord_flip() +
  labs(title = "Importancia de Variables - Random Forest Regresión",
       subtitle = "% de incremento en MSE al permutar la variable",
       x = "Variable", y = "% Incremento en MSE") +
  theme_minimal()
```

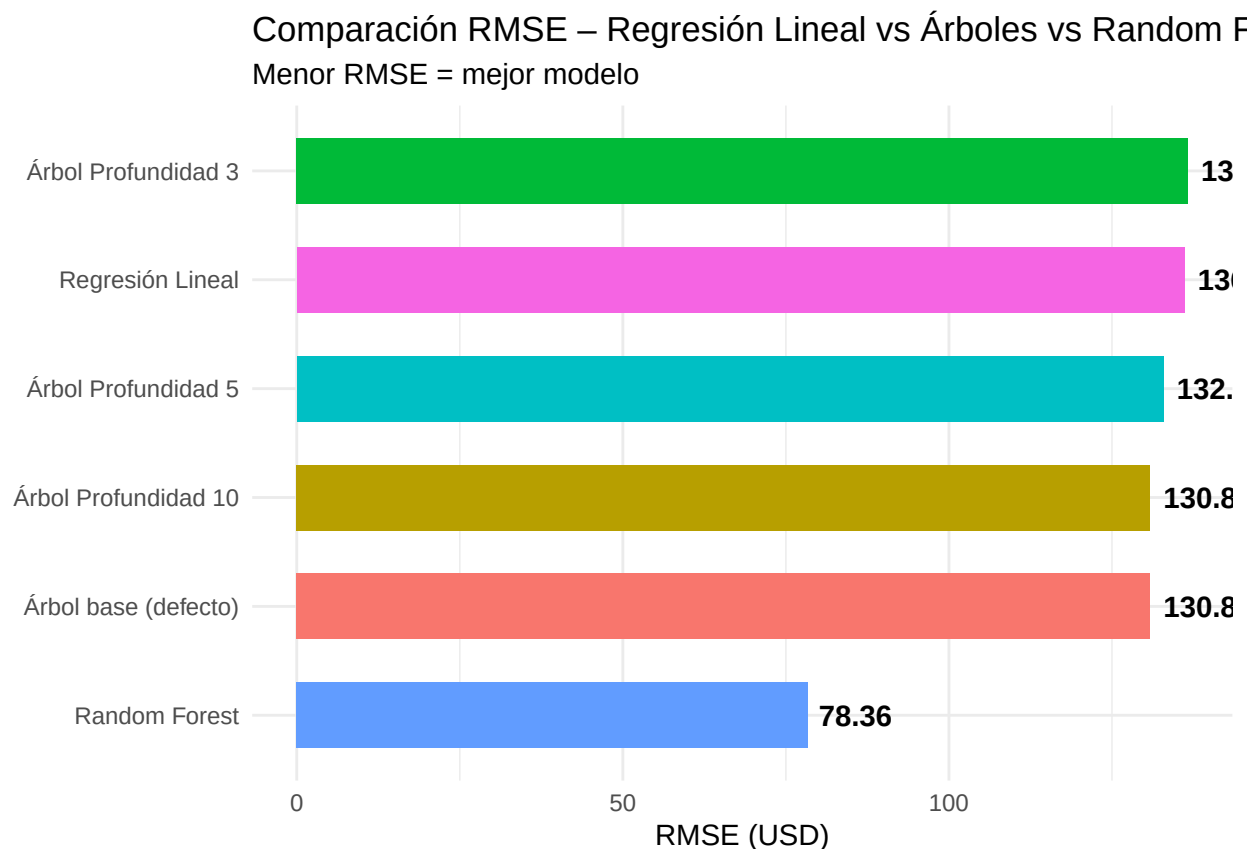


```
comparacion_reg_final <- data.frame(
  Modelo = c("Regresión Lineal",
             "Árbol base (defecto)",
             "Árbol Profundidad 3",
             "Árbol Profundidad 5",
             "Árbol Profundidad 10",
             "Random Forest"),
  RMSE = round(c(rmse_lineal, rmse_arbol,
                 rmse_prof3, rmse_prof5, rmse_prof10,
                 rmse_rf_reg), 2),
  MAE = round(c(mae_lineal, mae_arbol,
                 mae_prof3, mae_prof5, mae_prof10,
                 mae_rf_reg), 2),
  R2 = round(c(r2_lineal, r2_arbol,
                r2_prof3, r2_prof5, r2_prof10,
                r2_rf_reg), 4)
)
```

```
print(comparacion_reg_final)
```

```
##           Modelo  RMSE  MAE   R2
## 1  Regresión Lineal 136.11 94.95 0.3659
## 2  Árbol base (defecto) 130.84 90.38 0.4151
## 3  Árbol Profundidad 3 136.63 94.44 0.3622
## 4  Árbol Profundidad 5 132.89 91.93 0.3966
## 5  Árbol Profundidad 10 130.84 90.38 0.4151
## 6  Random Forest  78.36 46.96 0.8089
```

```
ggplot(comparacion_reg_final,
       aes(x = reorder(Modelo, RMSE), y = RMSE, fill = Modelo)) +
  geom_col(show.legend = FALSE, width = 0.6) +
  geom_text(aes(label = RMSE), hjust = -0.15, fontface = "bold") +
  coord_flip() +
  labs(title = "Comparación RMSE - Regresión Lineal vs Árboles vs Random Forest",
       subtitle = "Menor RMSE = mejor modelo",
       x = NULL, y = "RMSE (USD)") +
  theme_minimal()
```



El gráfico de comparación de RMSE muestra de forma contundente que Random Forest es el mejor algoritmo para predecir el precio por noche, con un RMSE de 78.36, casi 52 dólares menos que el segundo mejor modelo. Esto representa una mejora del 40% en el error respecto a los árboles individuales y del 42% respecto a la Regresión Lineal (\$136.11). Esta diferencia se explica porque Random Forest promedia las predicciones de 100 árboles entrenados con subconjuntos distintos de datos y variables, reduciendo drásticamente la varianza

sin aumentar el sesgo. La Regresión Lineal y el Árbol de Profundidad 3 quedan empatados como los peores modelos con RMSE.

Random Forest para CLASIFICACIÓN (precio_cat)

```
set.seed(123)
rf_clasificacion <- randomForest(
  precio_cat ~ .,
  data      = train_clas,
  ntree     = 100,
  mtry      = floor(sqrt(ncol(train_clas) - 1)),
  importance = TRUE
)

print(rf_clasificacion)
```

```
##
## Call:
## randomForest(formula = precio_cat ~ ., data = train_clas, ntree = 100,      mtry = floor(sqrt(ncol(
##           Type of random forest: classification
##           Number of trees: 100
## No. of variables tried at each split: 6
##
##           OOB estimate of  error rate: 16.39%
## Confusion matrix:
##           Economica Intermedia Cara class.error
## Economica      14749      1783  157  0.1162442
## Intermedia      2024     12871 1732  0.2258976
## Cara           236      1977 12734  0.1480565
```

```
pred_rf_clas <- predict(rf_clasificacion, newdata = test_clas)
cf_rf <- confusionMatrix(pred_rf_clas, test_resp_cat)
print(cf_rf)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Economica Intermedia Cara
## Economica      3775      507  49
## Intermedia      421     3223 466
## Cara           23      409 3193
##
## Overall Statistics
##
##           Accuracy : 0.8446
##           95% CI : (0.838, 0.851)
## No Information Rate : 0.3497
## P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7664
```

```
##
## McNemar's Test P-Value : 0.0001017
##
## Statistics by Class:
##
##           Class: Economica Class: Intermedia Class: Cara
## Sensitivity           0.8948           0.7787           0.8611
## Specificity           0.9291           0.8881           0.9483
## Pos Pred Value        0.8716           0.7842           0.8808
## Neg Pred Value        0.9426           0.8849           0.9390
## Prevalence            0.3497           0.3430           0.3073
## Detection Rate        0.3129           0.2671           0.2646
## Detection Prevalence  0.3589           0.3406           0.3004
## Balanced Accuracy      0.9120           0.8334           0.9047
```

```
cat("Accuracy: ", round(cf_rf$overall["Accuracy"] * 100, 2), "%\n")
```

```
## Accuracy: 84.46 %
```

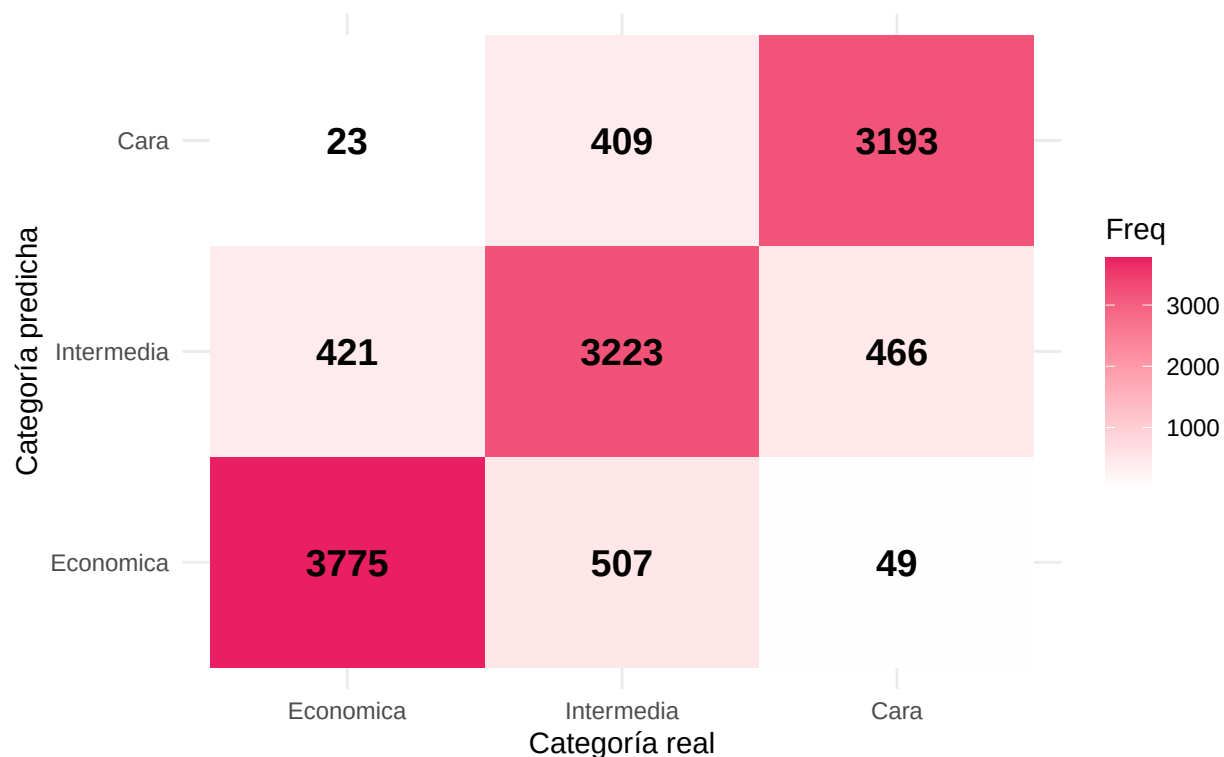
```
cat("Kappa:      ", round(cf_rf$overall["Kappa"], 4), "\n")
```

```
## Kappa:      0.7664
```

```
as.data.frame(cf_rf$table) %>%
  ggplot(aes(x = Reference, y = Prediction, fill = Freq)) +
  geom_tile() +
  geom_text(aes(label = Freq), fontface = "bold", size = 5) +
  scale_fill_gradient(low = "white", high = "#E91E63") +
  labs(title = "Matriz de Confusión - Random Forest",
       subtitle = "Filas = predicción | Columnas = valor real",
       x = "Categoría real", y = "Categoría predicha") +
  theme_minimal()
```

Matriz de Confusión – Random Forest

Filas = predicción | Columnas = valor real

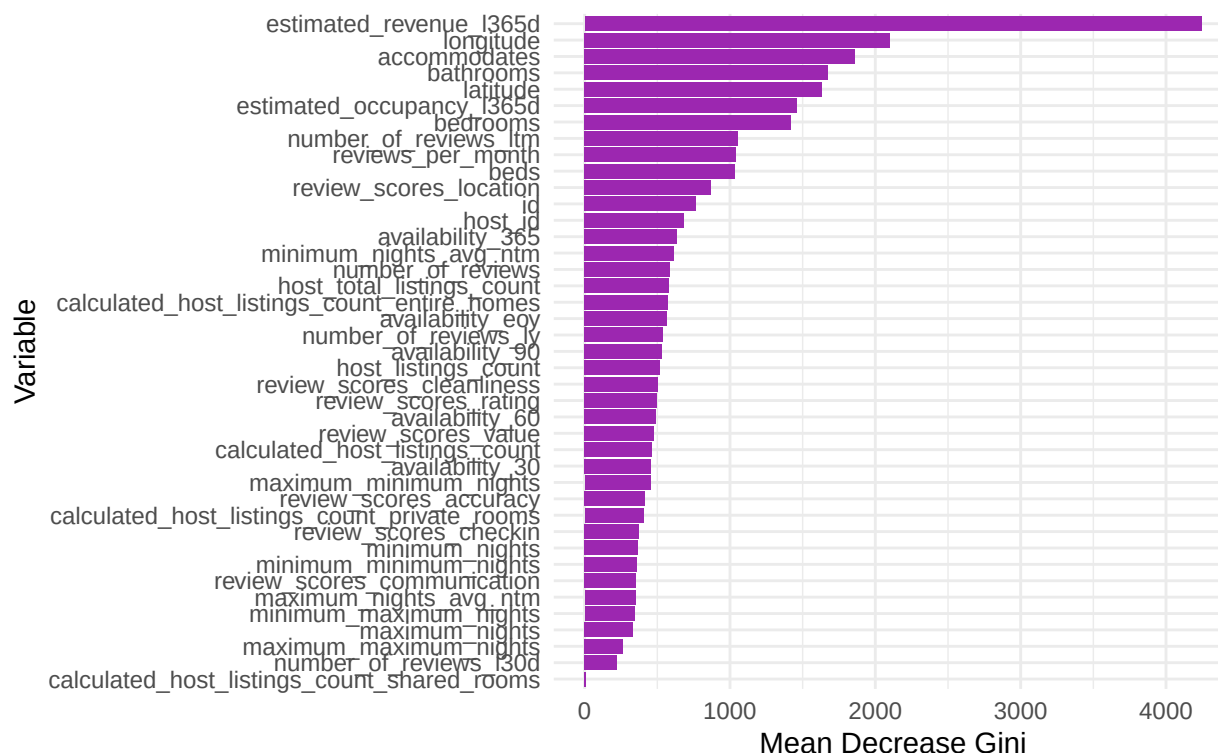


```
imp_rf_clas <- as.data.frame(importance(rf_clasificacion))
imp_rf_clas$variable <- rownames(imp_rf_clas)

ggplot(imp_rf_clas, aes(x = reorder(variable, MeanDecreaseGini),
                        y = MeanDecreaseGini)) +
  geom_col(fill = "#9C27B0") +
  coord_flip() +
  labs(title = "Importancia de Variables - Random Forest Clasificación",
       subtitle = "Reducción promedio del índice Gini",
       x = "Variable", y = "Mean Decrease Gini") +
  theme_minimal()
```

Importancia de Variables – Random Forest C

Reducción promedio del índice Gini



```
comparacion_clas_final <- data.frame(
  Modelo = c("Árbol base",
             "Árbol CV",
             paste0("Árbol tuneado (prof. ", mejor_prof_clas, ")"),
             "Random Forest"),
  Accuracy = round(c(cf$overall["Accuracy"],
                    cf_cv$overall["Accuracy"],
                    cf_tuneado$overall["Accuracy"],
                    cf_rf$overall["Accuracy"]) * 100, 2),
  Kappa = round(c(cf$overall["Kappa"],
                 cf_cv$overall["Kappa"],
                 cf_tuneado$overall["Kappa"],
                 cf_rf$overall["Kappa"]), 4)
)

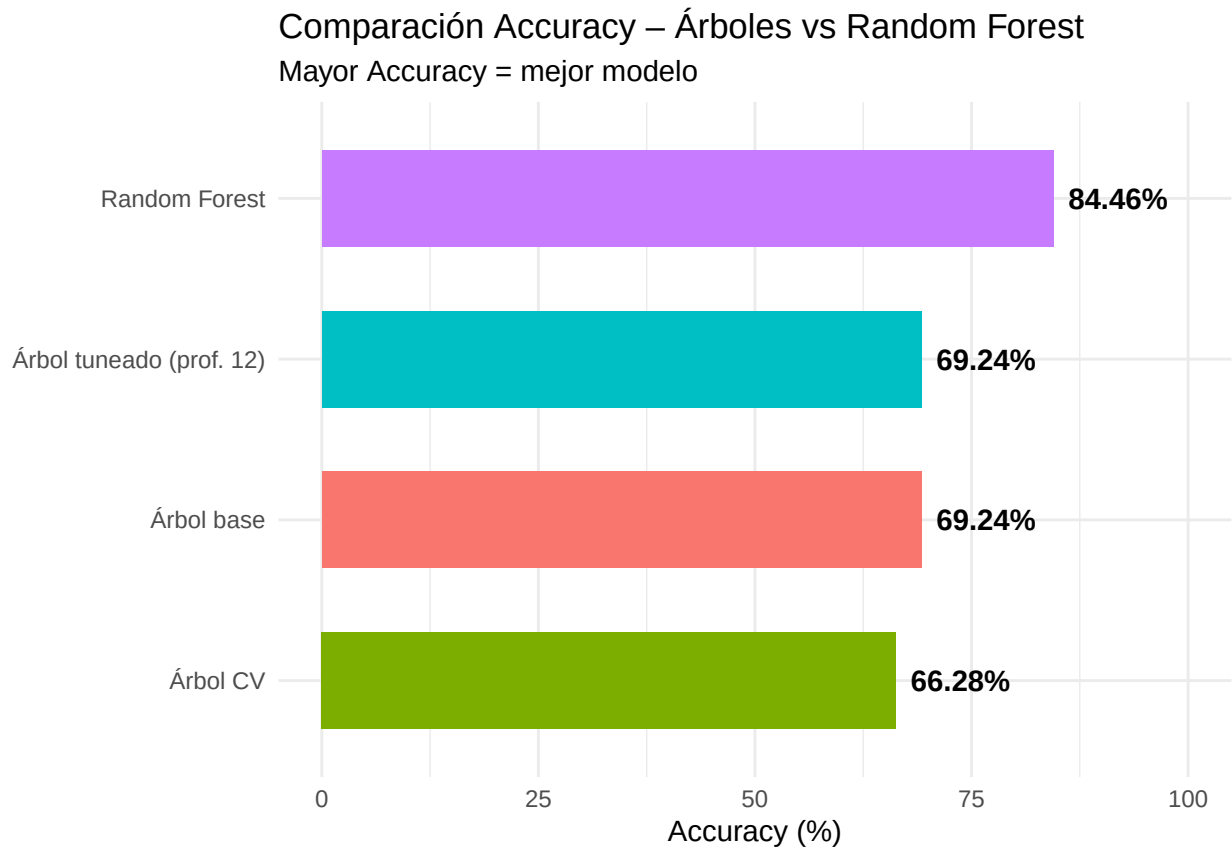
print(comparacion_clas_final)
```

```
##           Modelo Accuracy  Kappa
## 1         Árbol base    69.24 0.5380
## 2         Árbol CV     66.28 0.4944
## 3 Árbol tuneado (prof. 12) 69.24 0.5380
## 4         Random Forest 84.46 0.7664
```

```
ggplot(comparacion_clas_final,
  aes(x = reorder(Modelo, Accuracy), y = Accuracy, fill = Modelo)) +
```



```
geom_col(show.legend = FALSE, width = 0.6) +
geom_text(aes(label = paste0(Accuracy, "%")),
          hjust = -0.15, fontface = "bold") +
coord_flip() +
scale_y_continuous(limits = c(0, 100)) +
labs(title = "Comparación Accuracy - Árboles vs Random Forest",
      subtitle = "Mayor Accuracy = mejor modelo",
      x = NULL, y = "Accuracy (%)") +
theme_minimal()
```



Random Forest alcanza un Accuracy de 84.46%, superando por un margen muy amplio al mejor árbol individual (Árbol base y tuneado con 69.24%) y al árbol con validación cruzada (66.28%). Esto representa una mejora de 15 puntos porcentuales respecto a los árboles de decisión, lo cual es una diferencia prácticamente significativa para un sistema de recomendación de propiedades.